



ТЕХНИЧЕСКИ УНИВЕРСИТЕТ - ВАРНА
ФАКУЛТЕТ ПО ИЗЧИСЛИТЕЛНА ТЕХНИКА И АВТОМАТИЗАЦИЯ

Катедра
„Софтуерни и Интернет технологии”

Дипломна работа

Тема:

Проектиране и реализация на web приложение за
изучаване на чужди езици

Изготвил:
Сонер Кабилов
Специалност: СИТ
Факултетен номер: 20621507

Ръководител:.....
ас. д-р Даниела Петрова

Съдържание

Глава I – Увод	4
Глава II – Анализ на проблема и избор на технологии	6
1. Анализ на проблема	6
2. Обзор на съществуващите решения	7
3. Цели и изисквания	10
4. Задачи за разработка на дипломната работа	10
5. Избрани технологии.....	11
Глава III – Проектиране на системата	16
1. Изисквания на програмната система	16
2. Архитектура на системата.....	18
3. Логически модел	20
4. Диаграма на работния процес	23
5. Организация на данните	25
6. Макет на програмната система.....	30
Глава IV – Реализация на системата	34
1. Структура на данните	34
1.1. Аномалии и нормализации.....	34
1.2. Описание на таблиците от базата от данни	35
2. Описание на програмните модули	44
3. Структура и организация на потребителския интерфейс.....	49
3.1. Потребителска страница	49
3.2. Администраторска страница	65
3.3. Responsive дизайн.....	73
Глава V – Тестове и резултати.....	75
Глава VI – Заключение	80
Списък с фигури.....	81
Използвани източници.....	83
Приложение А: Изходен код.....	84

Глава I – Увод

Разработеното web приложение в настоящата дипломна работа и функционалностите, които са имплементирани в нея, дават възможност на потребителите да изучават чужди езици по интерактивен и интересен начин, използвайки когнитивния метод на обучение. Този метод се фокусира върху менталните процеси, участващи в ученето, като се набляга върху разбирането, разрешаването на проблеми и запомнянето на информация чрез повтаряне и допускане на грешки. По този начин хората могат не само да обогатят своите езикови умения, но и да подобрят своята памет и умствени способности.

Конкретното направление, към което е насочено приложението, е уроците да бъдат изградени под формата на игра. На потребителите се предоставят различни типове упражнения, които да подобрят техните умения за четене с разбиране, слушане и граматика. Освен типичните тестови въпроси, в системата е включена и възможност за изучаване чрез флаш карти, които също са се доказали като много ефективен метод за запомняне на нова информация.

За подрастващите в ученическа възраст този метод за обучение би могъл да се приложи като допълнение към тяхната училищна програма. Особено за деца с дефицит на вниманието, един такъв подход би могъл да се окаже изключително полезен начин те да възприемат по-лесно нова информация, отделяйки ги от рутината за учене посредством учебници. Игровият характер прави обучението по-забавно и мотивиращо, което е ключов фактор за задържане на интереса и подобряване на резултатите.

Освен в училищна среда, платформата е много ефективна и за хора, които нямат познание по даден език и не знаят от къде да започнат своето обучение. Бързите и кратки нива са подходящи дори за тези, които нямат възможност да отделят много време от своето ежедневие. Доказано е, че с дори десет минути обучение чрез подобен тип система може да бъде достатъчно човек да научи основни правила, граматика и лексика на езика за кратко време. Тази гъвкавост прави приложението изключително подходящо за заети хора, които искат да разширят своите умения без да жертват значително време и усилия.

С нарастващото потребление на електронните устройства, нараства и нуждата за електронно обучение. Доброто ниво на техническа грамотност на масовия потребител прави възможно областта на приложение на такъв тип система да обхване широк спектър от хора. Чрез доброто и правилно проектиране на приложението, масовите потребители ще имат лесен и удобен достъп от различни

места. Това включва не само достъп чрез компютър, но и чрез мобилни устройства, което допълнително улеснява използването на платформата в ежедневието на потребителите. По този начин, приложението не само улеснява изучаването на чужди езици, но и допринася за изграждането на една по-образована и свързана глобална общност.

Не съществува приложение на българския пазар, което да предлага изучаване на езици по подобен начин на български език. Това би могло да е голяма пречка за хората, които нямат познания по английски език и нямат друга алтернатива, по която да го научат. Дори и да желаят да изучават друг език, различен от английския, то те трябва да притежават добро ниво на езикова култура по английски език, за да имат възможността да започнат обучението си по друг чужд език. Друг техен недостатък е, че всяко от тези приложения най-често се фокусира върху конкретен метод за изучаване на езици. Добавяйки тези методи на едно място и разработвайки нови, също толкова ефективни, би било от изключително удобство за потребители, което би им спестило използването на различни системи.

Глава II – Анализ на проблема и избор на технологии

1. Анализ на проблема

Приложението има за цел да бъде използвано от голям брой потребители, чиито технически умения могат да бъдат на базово или средно ниво. Поради тази причина системата трябва да бъде максимално опростена, с интуитивен и удобен за потребителя дизайн, предразполагащ лека и удобна работа.

На първо място е необходимо да се реши на каква платформа следва да се изпълнява проекта. Възможните избори за реализация на подобна система включват:

- Уеб – базирана
- Мобилна
- Десктоп – базирана

За първоначална имплементация на приложението, най-подходящият вариант е да се разработи като уеб-базирана система, за да отговаря на изискването да може да бъде достъпно от всички устройства. Чрез своя responsive дизайн, уеб приложението би могло да се използва и от мобилни устройства, докато останалите варианти биха обхванали по-малък диапазон от потребители.

Тъй като става въпрос за уеб приложение, се очаква то да може да се използва от всички браузъри, удобни за всеки краен потребител. Тъй като ще се предлага на българския пазар, за повече удобство интерфейсът и част от данните, записани в базата данни, ще бъдат на български език.

Потребителският интерфейс на подобен тип система играе важна роля за цялостното усещане на потребителите и ги предразполага към научаване на нова информация. Топлите, приветливи цветове и добре подбраните шрифтове допринасят много за приятното и уютно усещане при използване на приложението, което да ги накара отново да се завърнат.

Основната задача на представената система е да се предостави различен и интересен метод за изучаване на чужди езици.

Следващата важна задача е да се определи каква функционалност ще изпълнява системата. Определя се всяка една функция, за да може на база на това да се изгради базата от данни. Дизайнът или още потребителският интерфейс, също е тясно свързано с функционалността. Базата от данни служи за съхранение на

информацията. Операциите четене, запис, модификация и изтриване ще се извършват единствено през приложението.

Също така трябва да се определи поведението на информационната система при възникване на грешки или правилно изпълнение на възникналото събитие. Възникването на грешки може да доведе до възпрепятстването на системата в тези ситуации. Не трябва да възникват ситуации, които биха довели до спиране на функционирането на уеб приложението.

Едно от най-важните неща е валидирането на данните, за да не се въвеждат некоректни данни в базата от данни. Данните, които се въвеждат трябва да отговарят на определени изисквания и условия.

В отделни части на системата ще се извеждат съобщения, информиращи потребителя дали извършената от него операция е била успешна или не.

2. Обзор на съществуващите решения

Приложенията за езиково обучение добиват все по-голяма популярност в световен мащаб. Чрез своя интерактивен метод на обучение, игрови и състезателен характер, те стават все по-привлекателни за желаещите да учат чужди езици.

Никое от най-известните системи за обучение на чужди езици обаче не предлага интерфейс за българоговорящи потребители. Друг техен недостатък е, че някои от тях са изцяло платени, а други предлагат ограничени функционалности за потребителите, които използват платформата безплатно.

- **Duolingo**

Duolingo е популярна платформа за езиково обучение, която предлага забавен и интерактивен начин за изучаване на нов език. Със своя игрови подход потребителите могат да извършват уроци, да печелят награди и да проследяват напредъка си, правейки изучаването на езици ангажиращо и достъпно за хора от всички възрасти.

Предимства:

- **Богата база от данни с множество езици и уроци към тях.**
Приложението предлага и обучение по измислени езици като Klingon и

High Valyrian, такива които са говорящи от малка група хора – Navajo, Hawaiian и др., а дори и латински, който е мъртъв език.

- **Голямо разнообразие от видове на упражнения.**
- **Състезателен характер.** Потребителите могат да участват в класации и да следят броя последователни дни, в които са преминали през поне едно ниво. Това може да стимулира потребителите да продължават да използват приложението за по-продължително време и да са по-постоянни в своето обучение.
- **Кратки уроци.** Бързите нива могат да се сместят лесно при натоварен график и да задържат вниманието на потребителите за по-дълго време.

Недостатъци:

- **Ограничения в безплатната версия.** Потребителите имат определен брой сърца, които се намалят при направена грешка, а при изхабяване на всички сърца, те нямат право да започнат нов урок. Това би могло да бъде много ограничаващо при започване на непознат език, тъй като съвсем естествено ще се допускат повече грешки и това ще забави прогреса на потребителите.
- **Един метод за изучаване на език.** Въпреки своето голямо разнообразие на упражнения, Duolingo предлага уроци, фокусиращи се предимно върху четенето и слушането.
- **Повтаряща се структура.** Някои потребители смятат, че повтарящият се характер на упражненията може да стане монотонен, потенциално да повлияе на дългосрочната ангажираност.

- **Glossika**

Glossika е иновативна платформа за изучаване на езици, която набляга на усвояването на естествен език чрез аудио обучение и повторение. Като се фокусира върху реални модели на изречения, Glossika помага на потребителите да изградят ефикасна лекота при говор и разбиране.

Предимства:

- **Фокус върху лекотата на говора.** Glossika набляга на усвояването на естествен език и моделите на изречения в реалния живот, като помага на потребителите да развият практически умения за говорене и слушане.

- **Система за повторение с интервали.** Използването на повторение с интервали гарантира, че потребителите запомнят лексиката и структурите по-ефективно във времето.
- **Аудио потапяне.** Аудио-базираното обучение на Glossika позволява на обучаемите да чуват родното произношение и интонация, подобрявайки своите способности за слушане и говорене.
- **Контекстуално обучение.** Като се фокусира върху цели изречения, а не върху изолирани думи, Glossika преподава език в контекст, което е полезно за разбирането на употребата и значението.

Недостатъци:

- **Цена.** Glossika е платена услуга, която може да бъде пречка за някои потребители в сравнение с безплатните алтернативи.
- **По-малко интерактивен.** Фокусът на платформата върху слушане и повтаряне на изречения може да бъде по-малко ангажиращ за потребители, които предпочитат интерактивни или игрови методи на обучение.
- **Ограничено визуално съдържание.** Glossika разчита предимно на аудио и текст, като липсват визуални помощни средства, които могат да помогнат с паметта и разбирането на някои учащи.
- **Техническа сложност.** Методът може да е по-малко интуитивен за абсолютно начинаещи, които могат да се възползват от по-структурирани, основни уроци по граматика и лексика.

• Babbel

Babbel е водещо приложение за изучаване на езици, предназначено да направи усвояването на нов език практично и ангажиращо. С експертно изработени уроци от лингвистични професионалисти, Babbel предлага интерактивни упражнения и диалози от реалния живот, за да помогне на потребителите да придобият бързо и ефективно умения при разговор.

Предимства:

- **Практически фокус.** Платформата набляга на диалозите от реалния живот и практическото използване на езика, като помага на потребителите да развият разговорни умения, подходящи за ежедневните ситуации.

- **Структурирани курсове.** Babbel предлага ясна и прогресивна структура, насочваща потребителите от начинаещи до по-напреднали нива с добре организирано съдържание.
- **Вникване в културните особености на езика.** Уроците често включват културен контекст, осигурявайки на учащите по-задълбочено разбиране на езика и употребата му в различни културни условия.

Недостатъци:

- **Абонаментна цена.** Babbel е услуга, базирана на абонамент, което може да е недостатък за потребителите, които търсят безплатни опции за изучаване на езици.
- **Ограничен избор на езици.** В сравнение с някои конкуренти, Babbel предлага по-малко възможности за избор на език, което може да не отговаря на нуждите на всички учащи.
- **По-малко игрови стил на обучение.** Подходът на Babbel е по-традиционен и по-малко игрови, което може да не е толкова ангажиращо за потребителите, които предпочитат по-подобно на игра учебно изживяване.
- **По-малък акцент върху практикуване на умения за говорене:** Докато Babbel включва упражнения за говорене, акцентът е повече върху четенето и писането, което може да не е достатъчно за развиване на силни разговорни умения.

3. Цели и изисквания

Целта на дипломната работа е да се изгради уеб-базирано приложение за изучаване на чужди езици, което да подпомогне обогатяването на езиковите знания и умствени способности на хората. Изискването е да се създаде приятен за използване потребителски интерфейс и правилно взаимодействие с базата данни.

4. Задачи за разработка на дипломната работа

Ключовите задачи, свързани с реализирането на дипломната работа са:

- **Проектиране на базата данни и потребителския интерфейс.** От изключително важно значение е правилното взаимодействие на базата данни и потребителския интерфейс с функционалностите на системата. Нужно е внимателно планиране как ще се представи информацията от

базата данни и по какъв начин ще се осъществяват корекциите, които ще се извършват в нея;

- **Проектиране на различни видове упражнения.** Системата трябва да предлага разнообразни упражнения, които не само да са полезни за потребителите, но и да задържат техния интерес, за да използват по-продължително време приложението;
- **Проектиране на допълнителни функционалности и помощни средства за обучение.** Добавяне на допълнителни методи за обучение, отделно от уроците под формата на игра. Повече обучителни методи на едно място, биха привлекли повече потребители, за които иначе би се наложило да използват различни платформи. Допълнителни функционалности като магазин би подпомогнало изучаването на чужди езици;
- **Проектиране на администраторски панел.** От голяма важност за поддържането на системата е наличието на администраторски панел, където да се извършват всички важни операции по добавяне, редактиране и изтриване на съдържание;
- **Тестване на системата.** Нужно е задълбочено тестване на операциите към базата данни, за да се предотвратят бъдещи проблеми, което да възпрепятства работата на администраторите и хубавото изживяване на обикновените потребители.

5. Избрани технологии

- **Node.js**



Фиг. 1 Лого на Node.js

Node.js е многофункционална и ефективна среда за изпълнение с отворен код, която революционизира начина, по който JavaScript се използва в разработката от страна на сървъра. Създаден на JavaScript машината на Chrome V8, Node.js позволява на разработчиците да изпълняват JavaScript код извън браузъра, позволявайки създаването на бързи, мащабируеми и високопроизводителни мрежови приложения. От въвеждането си през 2009 г., Node.js придоби значителна популярност в общността за разработка, превръщайки се в крайъгълен камък за модерното уеб развитие.

Една от най-забележителните характеристики на Node.js е неговата асинхронна и управлявана от събития архитектура. За разлика от традиционните server-side среди, които обработват заявки с помощта на множество нишки, Node.js работи на принципа на цикъл на събития с една нишка. Този неблокиращ механизъм позволява на Node.js да обработва многобройни едновременни връзки ефективно, което го прави особено подходящ за I/O-интензивни операции, като четене от бази данни или файлови системи. Тази архитектура не само подобрява производителността, но и подобрява скалируемостта на приложенията.

Node.js включва NPM (Node Package Manager), най-големият софтуерен регистър в света, хостващ милиони пакети, които предоставят кодови модули за многократна употреба за различни функционалности. Тази обширна библиотека ускорява разработката, като позволява на разработчиците да интегрират предварително изградени решения за общи задачи, вариращи от уеб рамки като Express.js до инструменти за манипулиране на данни и тестване.

- **HTML/CSS**



Фиг. 2 Лога на HTML и CSS

HTML (HyperText Markup Language) и CSS (Cascading Style Sheets) са основополагащите технологии в мрежата, позволяващи създаването и стилизирането на уебсайтове. HTML осигурява структурата и съдържанието на уеб страница, докато CSS е отговорен за нейното представяне, оформление и естетика. Заедно те формират гръбнака на уеб разработката, което прави възможно изграждането на визуално привлекателни и добре структурирани уебсайтове.

HTML, въведен през 1991 г, е стандартният език за маркиране, използван за създаване на структурата на уеб страниците. Състои се от поредица от елементи или тагове, които определят различни части от уеб страница, като заглавия, параграфи, връзки, изображения и списъци. Тези елементи са написани в обикновен текст и се интерпретират от уеб браузърите за правилно показване на съдържанието. HTML5, най-новата версия, въведе нови функции като семантични елементи, които подобряват значението и структурата на уеб съдържанието, подобрявайки достъпността и SEO (оптимизация за търсачки).

CSS, разработен през 1996 г., се използва за контрол на представянето на HTML елементи. Езикът позволява на разработчиците да дефинират стилове за различни елементи, включително цветове, шрифтове, разстояние и оформление. CSS е невероятно мощен, позволяващ на дизайнерите да създават адаптивни дизайни, които се адаптират към различни размери на екрана и устройства. Функции като Flexbox и CSS Grid революционизираха дизайна на оформлението, улеснявайки създаването на сложни, отзивчиви оформления, без да се разчита на таблици или плаващи елементи.

- **EJS**



Фиг. 3 Лого на EJS

EJS (Embedded JavaScript) е проста и мощна машина за шаблони, използвана в уеб разработката за генериране на HTML маркиране с обикновен JavaScript. EJS е проектиран да се интегрира безпроблемно с Node.js приложения, което го прави отличен избор за генериране на данни от страна на сървъра. Като позволява на разработчиците да вграждат JavaScript код в HTML шаблони, EJS улеснява създаването на динамични уеб страници, които могат ефективно да показват данни от сървъра.

EJS се отличава със своята простота и гъвкавост. Една от основните му функции е възможността за вграждане на JavaScript код директно в HTML чрез специални тагове. Този подход позволява на разработчиците да пишат логика директно в своите шаблони, което улеснява представянето на данни и включването на частични шаблони, които да бъдат многократно преизползвани.

EJS е особено подходящ за използване с Node.js. В типично Node.js приложение EJS може да се използва за генериране на HTML страници на сървъра, преди да ги изпрати на клиента. Това изобразяване от страна на сървъра (SSR) подобрява производителността чрез намаляване на количеството JavaScript, което трябва да се изпълни в брауъра, което води до по-бързо зареждане на страниците и по-добро SEO. Интегрирането на EJS с Node.js е лесно и често изисква само няколко реда код за настройка във рамка като Express.js.

- **JavaScript**



Фиг. 4 Лого на JavaScript

JavaScript е динамичен език за програмиране на високо ниво, който се е превърнал в крайгълен камък на съвременното уеб развитие. Създаден през 1995 г, езикът е проектиран да добави интерактивност към статичните HTML страници. През годините той се развива значително, очертавайки се като многофункционален и мощен език за програмиране.

Един от най-завладяващите атрибути на JavaScript е неговата повсеместност. Това е единственият език за програмиране, който се поддържа от всички основни уеб браузъри, което го прави незаменим за уеб разработка от страна на клиента. JavaScript позволява на разработчиците да създават интерактивни елементи като валидации на формуляри, динамични актуализации на съдържанието, анимации и др. Неговата гъвкавост се простира отвъд браузъра; с навлизането на среди като Node.js, JavaScript вече може да се използва за разработка от страна на сървъра, отваряйки безброй възможности за изграждане на мащабируеми и високопроизводителни приложения.

- **MySQL**



Фиг. 5 Лого на MySQL

MySQL е една от най-популярните и широко използвани системи за управление на релационни бази данни (RDBMS) с отворен код. Разработен в средата на 90-те години от шведската компания MySQL AB и сега собственост на Oracle Corporation, MySQL се е превърнал в основен компонент на много уеб приложения и онлайн услуги поради своята надеждност, лекота на използване и стабилен набор от функции.

Основната функционалност на MySQL е съхранението, извличането и управлението на данни с помощта на език за структурирани заявки (SQL). Една от основните му силни страни е неговата производителност и ефективност. MySQL е проектиран да обработва голям обем транзакции и големи набори от данни, което го прави идеален избор за приложения, вариращи от малки уебсайтове до

широкомащабни корпоративни системи. Способността му да управлява множество бази данни едновременно позволява гъвкави и мащабируеми решения за управление на данни.

Една от забележителните характеристики на MySQL е неговата гъвкавост и лекота на интегриране с различни програмни езици и платформи. Той безпроблемно се интегрира с популярни езици за програмиране като PHP, Python, Java и Node.js, което го прави предпочитан избор за разработчиците, целящи да създадат динамични уеб приложения. Съвместимостта на MySQL с различни операционни системи, включително Windows, Linux и macOS, допълнително подобрява неговата адаптивност.

- **Stripe**



Фиг. 6 Лого на Stripe

Stripe е водеща технологична компания, която предоставя икономическа инфраструктура за интернет, специализирана в обработка на онлайн плащания и финансови услуги. Основана през 2010 г., Stripe бързо се разраства, за да се превърне в предпочитан портал за плащане за фирми от всякакъв размер, предлагайки набор от инструменти за улесняване на безпроблемни онлайн транзакции. Неговата лесна за разработчици платформа поддържа широк набор от методи на плащане, което го прави основен инструмент за електронна търговия и други интернет базирани бизнеси и дава възможност на разработчиците да тестват извършване на транзакции чрез своите фиктивни карти.

Глава III – Проектиране на системата

1. Изисквания на програмната система

Разработването на уеб приложение е предназначено за изграждане на система за изучаване на чужди езици чрез различни методи за обучение. Тази система е предвидена да предостави динамичен изглед, с който потребителят да си взаимодейства, и да изгради надеждна връзка с базата данни за осигуряване на правилната работа на приложението.

Избраните технологии за разработка на дипломния проект, са подбрани, така че да бъдат подходящи за работа с множество заявки към базата данни. Неблокиращият механизъм на работа на Node.js гарантира, че няма да се получи блокиране на нишка, поради голям брой изпратени заявки, а добрата му съвместимост с други front-end технологии позволява разработката на едно по-динамично уеб приложение.

Бизнес правилата са специфични насоки или ограничения, които диктуват поведението и работата на софтуерното приложение. Те са от съществено значение, за да се гарантира, че приложението отговаря на бизнес изискванията и е в съответствие с организационните политики. Някои от основни бизнес правила в приложението са:

- **Управление на профили**

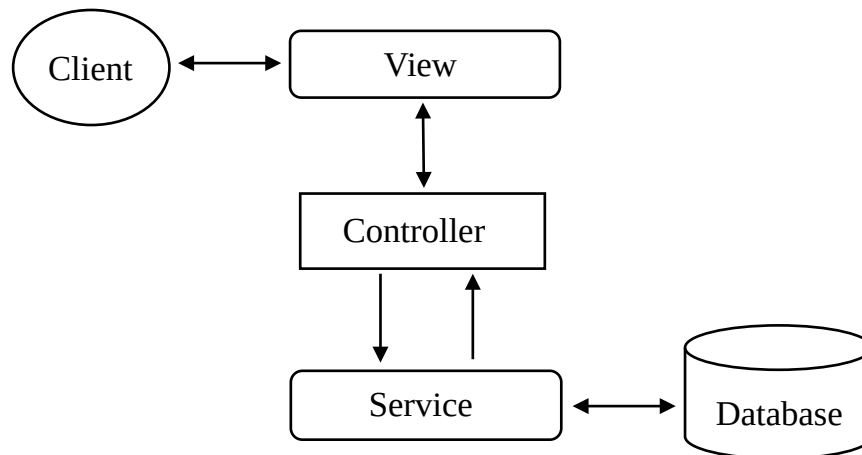
- Потребителската страница може да бъде достъпена от обикновени потребители и администратори;
- Администраторската страница може да бъде достъпена единствено от потребители със специални права;
- Администраторските профили могат да бъдат създавани единствено от главния администратор;
- Обикновените потребители могат да влязат в системата единствено през формата за вход за потребители, а администраторите единствено през формата за вход за администратори;
- Всички въведени данни трябва да бъдат валидирани за коректност.

- **Достъп до уроци**

- Всеки потребител има достъп до безплатните уроци;
- Единствено потребители с активен абонамент имат достъп до платените уроци;

- Потребителите имат достъп единствено до уроците, до които са прогресирали;
- Урок се счита за преминат единствено след успешно преминаване на всички упражнения;
- При извършване на грешка в упражнение, на потребителите трябва да се предоставят други методи за научаване на упражнението.
- **Извършване на покупка**
 - При извършване на покупка с монети, потребителят трябва да има достатъчна наличност;
 - При извършване на покупка с банкова карта, данните за картата трябва да бъдат коректни;
 - Всеки потребител трябва да има достъп до преглед на извършените покупки.
- **Разширяване на съдържанието в системата**
 - Единствено администраторите имат право да извършват операции за добавяне, редактиране и изтриване на съдържанието;
 - Безплатен урок може да се добави единствено към последно добавен безплатен раздел;
 - Може да съществува само един платен раздел от тип за даден курс;
 - Опциите за отговор и правилният отговор за упражнение трябва да са въведени коректно, за да бъде успешно добавянето на упражнението.
- **Коригиране и изтриване на съдържанието в системата**
 - При изтриване на безплатен или платен урок трябва да се извърши редакция на поредността на останалите уроци, както и на прогреса на потребителите. Изтритите уроци се отбелязват с поредност ,0‘ в базата данни;
 - Изтриването на безплатен раздел може да се осъществи единствено ако няма налични уроци в него. Последователността на изтрития раздел се редактира на ,0‘ в базата данни.

2. Архитектура на системата



Фиг. 7 Архитектура на системата

N-слойната архитектура, често наричана многослойна архитектура, е преобладаващ модел на проектиране в софтуерното инженерство, използван за създаване на мащабируеми и поддържаеми корпоративни приложения. В текущата разработка архитектурата разделя приложението на четири отделни слоя: слой на изгледа (view), слой на контролера (controller), бизнес логика (service) и слой на хранилище (repository). Всеки слой е отговорен за конкретни задачи, като насърчава разделянето на проблемите и подобрява цялостната структура на приложението.

- View

Слоят на изгледа, известен още като презентационен слой, е най-горният слой на N-слойната архитектура. Той е отговорен за взаимодействието с потребителя чрез показване на данни и улавяне на потребителски входове. Този слой включва компоненти на потребителския интерфейс като HTML, CSS, JavaScript. Чрез изолиране на елементите на потребителския интерфейс могат да бъдат направени промени в дизайна, без да се засяга основната бизнес логика или механизмите за достъп до данни.

- Controller

Слоят на контролера действа като посредник между слоя на изгледа и бизнес логиката. Той обработва входящите данни от изгледа, изпраща данните за обработка към бизнес логиката и връща подходящи отговори обратно към изгледа. Контролерите управляват потока на приложението, като гарантират, че потребителските въведени данни са валидирани и насочени към подходящите

методи за обслужване. Това разделяне позволява на бизнес логиката да остане отделена от логиката на представяне, правейки приложението по-модулно. Технологията, която се използва в разработката на дипломния проект, е Express.js в Node.js.

- Service

Под слоя на контролера се намира сервизният слой, известен също като слой на бизнес логиката. Този слой капсулира основната функционалност и бизнес правилата на приложението. Той обработва данни от нивото на контролера, прилага бизнес правила и гарантира, че операциите се извършват правилно. Чрез изолиране на бизнес логиката, промените в основната функционалност могат да бъдат внедрени и тествани независимо от потребителския интерфейс и логиката за достъп до данните от базата данни.

- Repository

Слоят на хранилището, известен също като слой за достъп до данни, управлява взаимодействието с базата данни или други източници на данни. Той осигурява абстракция върху механизмите за съхранение на данни, позволявайки на слоя на услугата да извършва операции с данни, без да е необходимо да знае спецификата на технологията или структурата на базата данни. Този слой обработва CRUD (създаване, четене, актуализиране, изтриване) операции и гарантира целостта и сигурността на данните. Технологията, която се използва за взаимодействие с базата данни в разработката на дипломния проект, е MySQL Server и MySQL Workbench.

N-слойната архитектура е мощен модел за проектиране, който осигурява ясна и организирана структура за разработване на сложни приложения. Чрез разделяне на приложението на отделни слоеве, то подобрява поддръжката, скалируемостта и възможността за повторна употреба на компоненти от различни слоеве, които да могат да бъдат използвани в различни части на приложението повторно. Това го прави предпочитан избор за изграждане на стабилни и ефективни корпоративни решения. Този архитектурен подход гарантира, че всеки слой може да се развива независимо, което води до по-гъвкава и адаптивна система като цяло.

3. Логически модел

След като са определени основните технологии и архитектурата, които ще се използват при изграждане на дипломния проект, могат да се определят и функционалностите, които ще се разработят в системата.

Чрез изготвяне на логически модел на системата може да се представи абстрактно структурата, поведението и взаимодействието на компонентите на дадено приложение. Той описва как системата функционира и как различните части на системата си взаимодействат помежду си и с външни фактори. Логическият модел не се фокусира върху физическите или техническите детайли на системата, а по-скоро върху концептуалните аспекти.

Основните компоненти на логическия модел са:

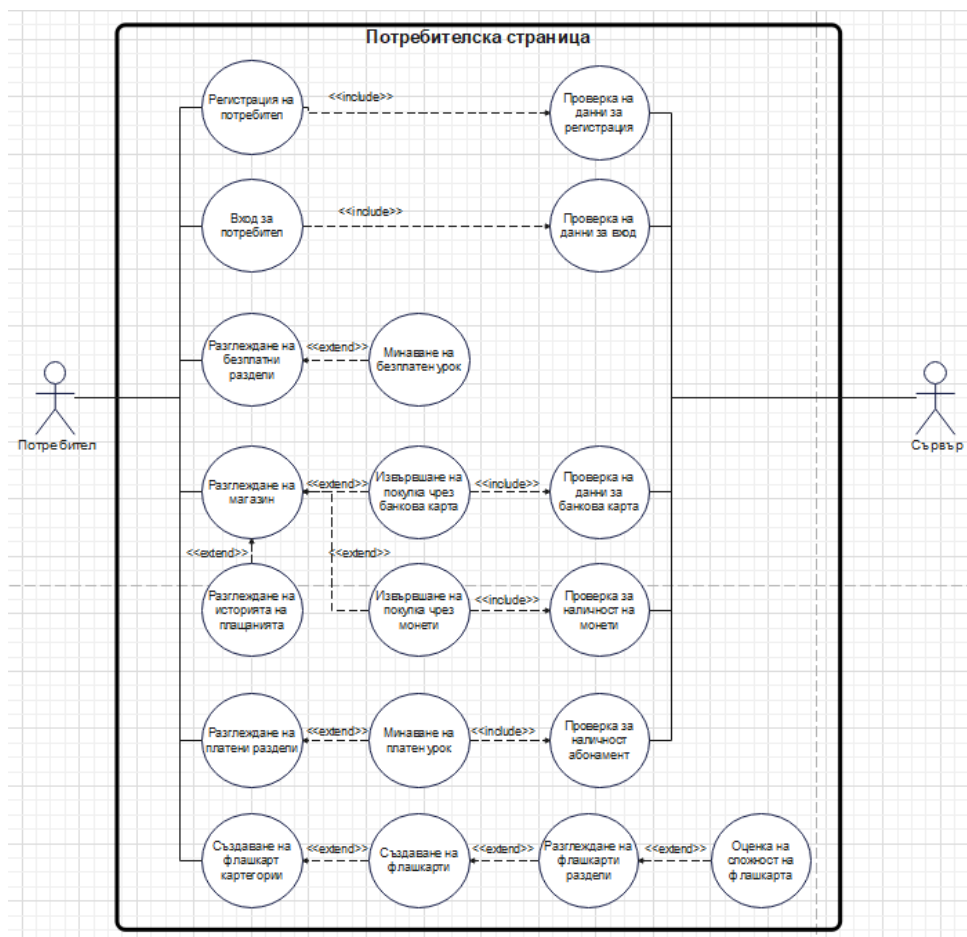
1. **Елементи/компоненти** - основните части на системата, които изпълняват специфични функции;
2. **Връзки** - описват как елементите на системата си взаимодействат помежду си, включително потока на информация или контрол;
3. **Интерфейси** - начините, по които системата комуникира с външни системи или потребители;
4. **Процеси** - описват последователността от стъпки или операции, които системата извършва, за да изпълни своите функции;
5. **Данни** - информацията, която се използва и обработва от системата.

Проектът е разделен на две части – потребителска и администраторска. Всяка една от двете части притежава отделен прозорец за вход и регистрация.

Функционалностите, до които имат достъп обикновените потребители, са:

- Вход и регистрация в системата за обикновени потребители;
- Преглеждане и минаване на безплатни уроци;
- Преглеждане и минаване на платени уроци, ако потребителят има закупен абонамент;
- Създаване и преглеждане на флашкарти и категории за флашкарти;
- Преглед на магазин и извършване на покупка. Предметите в магазина ще могат да бъдат закупени по два начина – с монети, спечелени от преминаването на уроци, и чрез плащане с банкова карта;
- Преглед на историята на покупките, извършени през магазина;
- Изход от системата.

Функционалностите на обикновения потребител могат да бъдат представени чрез следната UML Use Case диаграма:

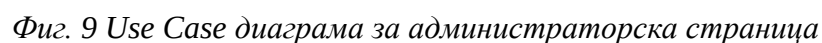


Фиг. 8 Use Case диаграма за потребителска страница

Администраторите имат достъп до всички функционалности на потребителската страница, включително следните допълни функционалности:

- Вход за администратор;
- Създаване на администраторски профил. Единствено главният администратор ще има право да създава профили на други администратори;
- Добавяне на нови курсове. Няма да бъде възможно добавянето на нов курс, ако такъв вече съществува;
- Преглед, редакция и изтриване на добавени курсове;
- Добавяне, преглед, редакция и изтриване на раздели към създадените курсове;

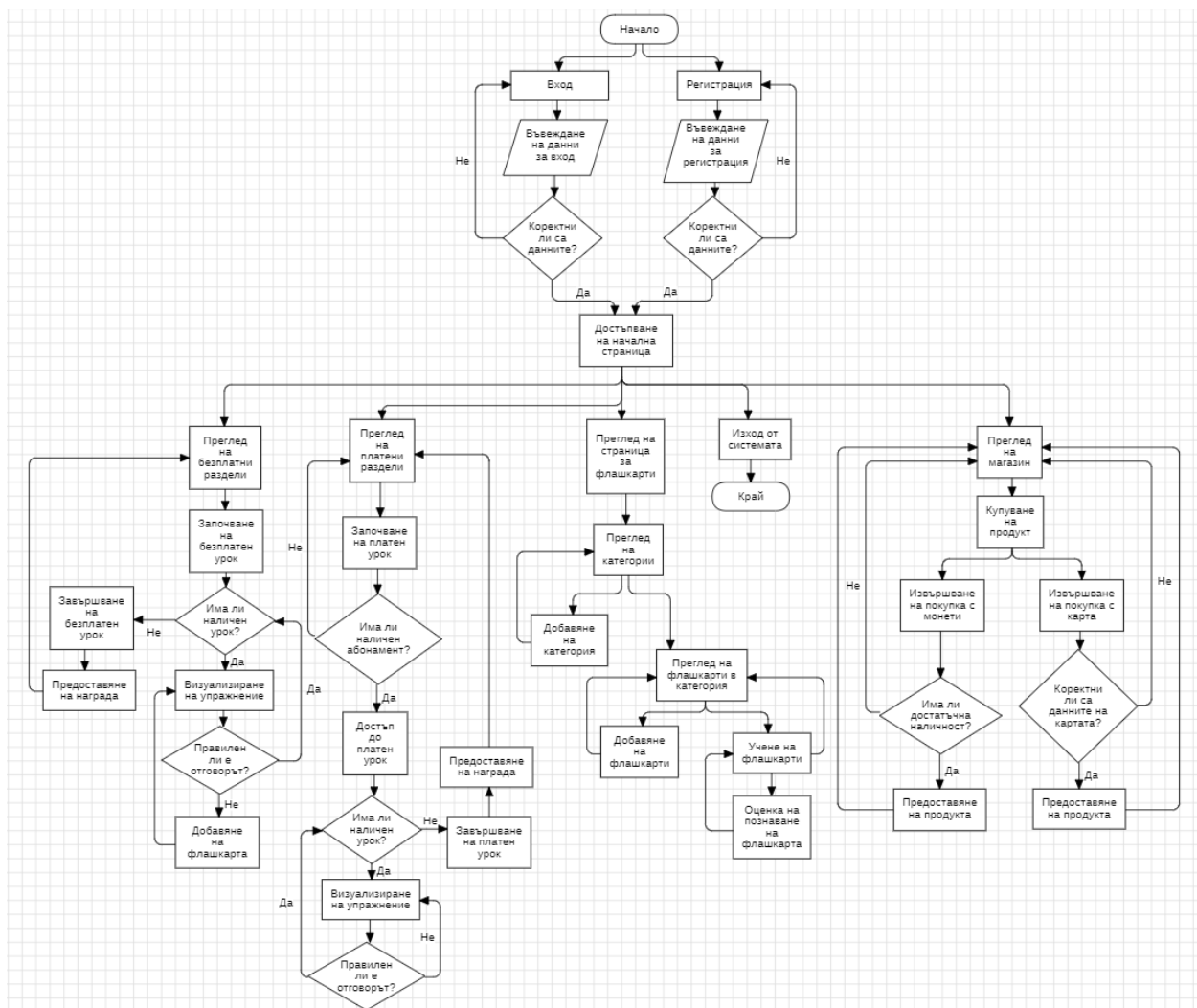
- Функционалностите на администраторския панел могат да бъдат представени чрез следната UML Use Case диаграма:



4. Диаграма на работния процес

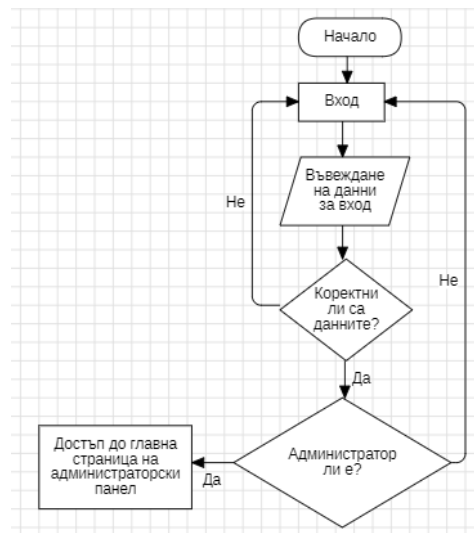
Диаграмата на работния процес е визуално представяне на стъпките или процесите, които са необходими за изпълнение на дадена задача или работа. Тя показва последователността на действията, потока на информация и взаимодействията между различните елементи и участници в процеса. Тези диаграми са полезни за разбиране, анализ и оптимизиране на работните процеси в различни контексти – бизнес, производство, софтуерно инженерство и др.

Работният процес на потребителската страница може да се представи чрез следната диаграма:

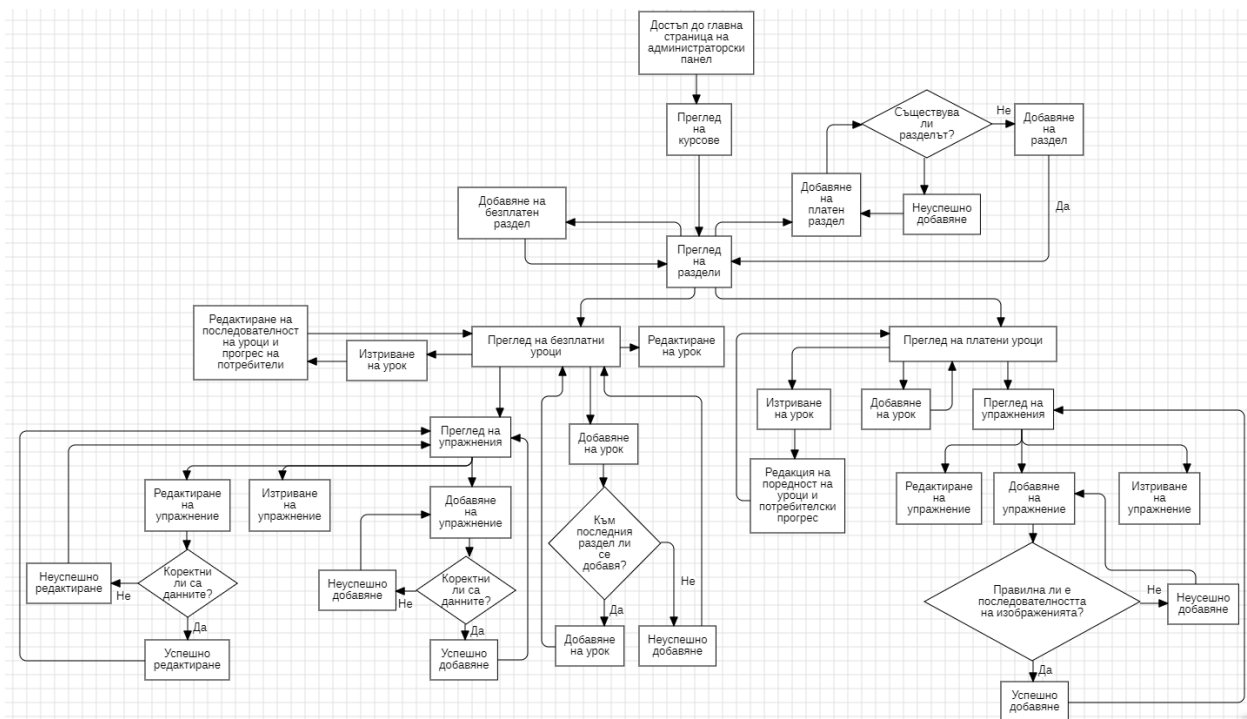


Фиг. 10 Flowchart диаграма за потребителска страница

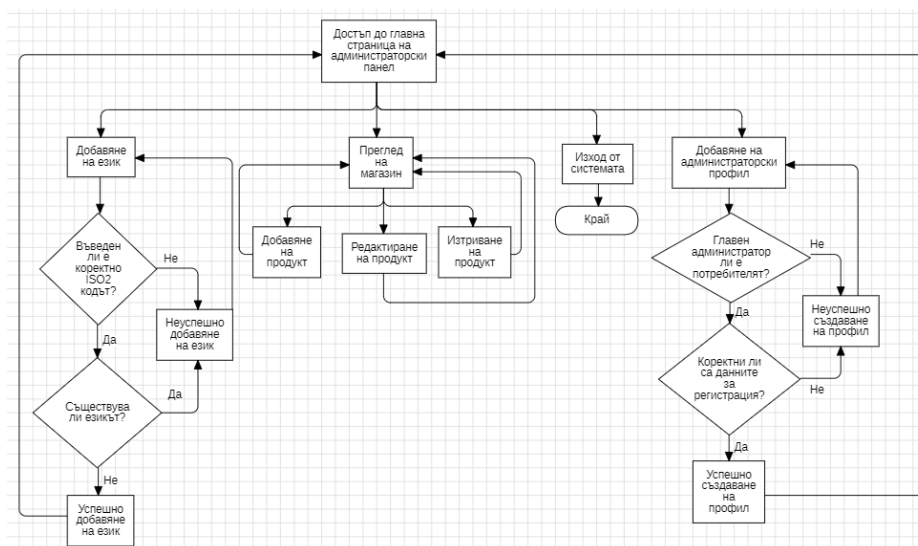
Работният процес на администраторската страница може да се представи чрез следните диаграми:



Фиг. 11 Flowchart диаграма за вход на администратор



Фиг. 12 Flowchart диаграма за преглед на курсове, раздели, уроци и упражнения



Фиг. 13 Flowchart диаграма за функционалности на администратор

5. Организация на данните

Разглежданата система ще съхранява данни в база данни MySQL. Начинът на съхранение на данните в базата се нарича модел. За да се определи как да бъде създаден този модел, е необходимо предварително да се дефинират целите на системата и нейните функционалности. Организацията и анализът на данните са ключови стъпки за изграждане на цялостна и ефективна система.

Моделът на представяне на данните в базата е релационен, тъй като данните са представени в таблична форма. Таблиците могат да бъдат логически свързани помежду си. Всяка таблица съдържа множество записи, а всеки запис се състои от полета (атрибути). Всеки запис притежава уникален идентификатор, наречен първичен ключ (primary key). Освен него съществуват и така наречените вторични ключове (foreign key), които се използват за връзки между таблиците, но не са задължителни. Тези връзки могат да бъдат от следните видове:

- 1:1 (едно към едно) – един запис от първата таблица се свързва с един запис от втората таблица.
- 1:m (едно към много) – един запис от първата таблица се свързва с един или повече записи от втората таблица; един запис от втората таблица се свързва само с един запис от първата таблица.

- m:n (много към много) – един запис от първата таблица се свързва с един или повече записи от втората таблица; един запис от втората таблица се свързва с един или повече записи от първата таблица.

Базата от данни, която се разработва, е сървърна. Тя се намира на отделна машина, до която имат достъп всички клиенти. За съхранение на информацията връзката към таблиците в базата данни се осъществява посредством заявки към сървър. След получаване на заявка, сървърът се свързва с базата данни, за да изпълни необходимата операция, и връща отговор на клиента за успешното или неуспешно изпълнение на заявката. От своя страна, базата от данни съхранява различни по характер данни, необходими за основните функционалности на разработваното приложение.

Следва да се създаде концептуална схема на базата данни. Това е начин на представяне на данните, без да се навлиза в детайли.

Проектираната база данни ще се състои от следните таблици и полета:

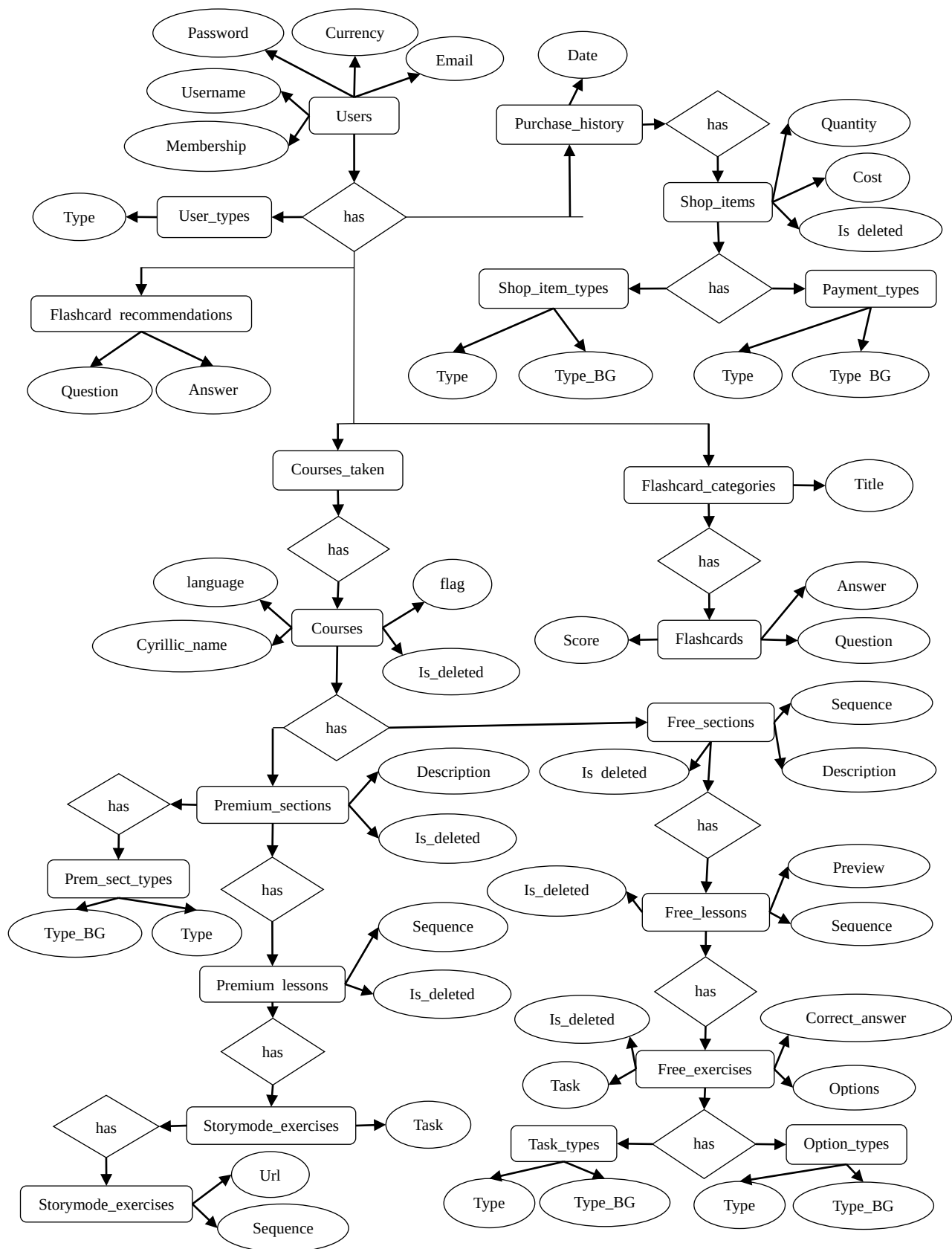
- Users – таблица за регистрираните потребители в системата
 - o id – първичен ключ;
 - o username – потребителско име;
 - o password – парола;
 - o email;
 - o currency – налична валута;
 - o membership – абонамент.
- User_types – таблица за видовете потребители
 - o id - първичен ключ;
 - o type – тип на потребител.
- Courses – таблица за налични курсове
 - o id - първичен ключ;
 - o language – език;
 - o cyrillic_name – език, изписан на кирилица (български);
 - o flag – флаг на държавата;
 - o is_deleted – налично/изтрито.
- Courses_taken – таблица за започнати курсове от потребител
 - o id - първичен ключ;
 - o on_lesson – текущ урок (безплатен);
 - o on_storymode_lesson – текущ урок („Истории“).
- Free_sections – таблица за безплатни раздели
 - o id - първичен ключ;

- o sequence – поредност на раздел;
 - o description – описание на раздел;
 - o is_deleted - налично/изтрито.
- Free_lessons – таблица за безплатни уроци
 - o id - първичен ключ;
 - o sequence – поредност на урок;
 - o preview – преглед на урок;
 - o is_deleted - налично/изтрито.
- Free_exercises – таблица за безплатни упражнения
 - o id - първичен ключ;
 - o task – задание на упражнение;
 - o options – възможни отговори;
 - o correct_answer – правилен отговор;
 - o is_deleted - налично/изтрито.
- Task_types – таблица за тип на заданието на безплатно упражнение
 - o id - първичен ключ;
 - o type – тип на заданието;
 - o type_bulgarian - тип на задачите, изписано на български език.
- Option_types – таблица за тип на отговорите на безплатно упражнение
 - o id - първичен ключ;
 - o type – тип на заданието;
 - o type_bulgarian - тип на отговор, изписано на български език.
- Premium_sections – таблица за платени раздели
 - o id - първичен ключ;
 - o description – описание на раздел.
- Premium_section_types – таблица за типове платени раздели
 - o id - първичен ключ;
 - o type - тип на платен раздел;
 - o type_bulgarian - тип на платен раздел, изписан на български език.
- Premium_lessons – таблица за платени уроци
 - o id – първичен ключ;
 - o sequence – поредност на урок;
 - o is_deleted - налично/изтрито.
- Storymode_exercises – таблица за упражнения „Истории“
 - o id – първичен ключ;
 - o task – задание на упражнение.
- Storymode_images - таблица за снимки на упражненията „Истории“
 - o id – първичен ключ;

- o url – път до снимка на упражнение „Истории“;
 - o sequence – поредност на снимка.
- Flashcard_categories – таблица за категории на флашкарти
 - o id – първичен ключ;
 - o title – заглавие на раздел.
- Flashcards – таблица за флашкарти
 - o id – първичен ключ;
 - o question – въпрос за флашкарта;
 - o answer – отговор за флашкарта;
 - o score – оценка за познаване на флашкарта.
- Flashcard_recommendations – таблица за препоръчани флашкарти
 - o id – първичен ключ;
 - o question – въпрос за флашкарта;
 - o answer – отговор за флашкарта.
- Shop_items – таблица за продукти в магазина
 - o id – първичен ключ;
 - o quantity – количество на продукта;
 - o cost – цена на продукта;
 - o is_deleted – налично/изтрито.
- Purchase_history – таблица за история на покупките
 - o id – първичен ключ;
 - o date – дата на покупка.
- Payment_types – таблица за тип на плащане
 - o id – първичен ключ;
 - o type – тип на плащане;
 - o type_bulgarian - тип на плащане, изписано на български език.
- Shop_item_types – таблица за тип на продукт в магазина
 - o id – първичен ключ;
 - o type – тип на продукт;
 - o type_bulgarian - тип на продукт, изписано на български език.

Използването на база данни в системата има за цел да осигури съхранение на данни, като позволява на потребителите да достъпват, съхраняват и извличат информацията си. Базата данни предлага по-сигурен начин за съхранение на данните, защото ограничава достъпа и защитава от неоторизиран достъп.

От идейния замисъл на таблиците и полетата в базата данни, може да се изведе следната ER (Entity-relationship) диаграма представена на Фигура 3.6:



Фиг. 14 Концептуален модел на БД

Изградената база данни е релационна. Предимствата ѝ включват лесен концептуален модел, благодарение на релационната схема, която точно съответства на модела.

Като недостатък може да се посочи сложността, тъй като базата данни може да съдържа множество таблици и връзки между тях, което може да направи някои операции по-бавни и по-сложни.

Основни функции на базата данни:

- Create заявки – заявки за създаване на базата данни и различните таблици в нея;
- Insert заявки – заявки за добавяне на записи в конкретна таблица, като се проверява дали записите имат уникален идентификатор (ID), дали полетата не са празни или нулеви, и дали отговарят на съответния тип на колоната. Важно е да се спазват правилата, зададени при създаването на таблицата;
- Select заявки – заявки за извличане на данни от конкретни таблици според зададени критерии;
- Update заявки – заявки за промяна на съдържанието на вече съществуващ запис в дадена таблица, като трябва да се посочи кой запис ще бъде обновен;
- Delete заявка – заявки за изтриване на записи от дадена таблица, като трябва да се посочи кой запис ще бъде изтрит.

6. Макет на програмната система

На база извършения анализ бяха разработени няколко прототипа за най-основните страници в приложението. Бяха подбрани приятни и топли цветове, които да не дразнят потребителите, но и да ги предразположат към обучение. Шрифтът, който беше подбран, също отговаря на темата на разработеното приложение с лесно четим размер. Основната цел е потребителите да се навигират лесно в приложението, без да се затруднява тяхната работа.

Следните прототипи са разработени във Figma:

Вход

Потребителско име

Парола

Нямате профил? [Регистрация](#)

Вход

Фиг. 15 Прототип на страница за вход

Уроци

Уроци Флашкарти Магазин

100

Английски

Френски

Добави курс

Безплатни раздели

Раздел 1

описание

Начало

Раздел 2

описание

Начало

Раздел 3

описание

Начало

Урок 1

Начало

Фиг. 16 Прототип на страница за визуализиране на уроци

Упражнение

✕

Напишете на английски: "момче и котка"

a

boy

cat

I

woman

a

man

you

and

Проверка

Фиг. 17 Прототип на страница за визуализиране на упражнения

Флашкарти

Уроци Флашкарти Магазин

100

Добавяне на категория

Английски

Думи

Английски

Преглед Слаби страни

Добавяне на флашкарта Препоръчани флашкарти

0	B cat	O котка	
5	B boy	O момче	

Фиг. 18 Прототип на страница за флашкарти

Администраторски панел	
Администратор	Раздели
Раздели	Английски ▼
Добавяне на администраторски профил	Безплатни раздели
Добавяне на курс	Раздел 1 – Използвайте основни фрази
Добавяне на предмет в магазина	Раздел 2 – Поздравявайте хората
Изход	Платени раздели
	Истории
	Френски ►

Фиг. 19 Прототип на администраторски панел

Глава IV – Реализация на системата

1. Структура на данните

Реализацията на базата от данни може да стане по два начина: чрез използване на потребителския интерфейс MySQL Workbench или чрез използване на SQL команди (заявки). В настоящата разработка се използва MySQL Workbench.

1.1. Аномалии и нормализации

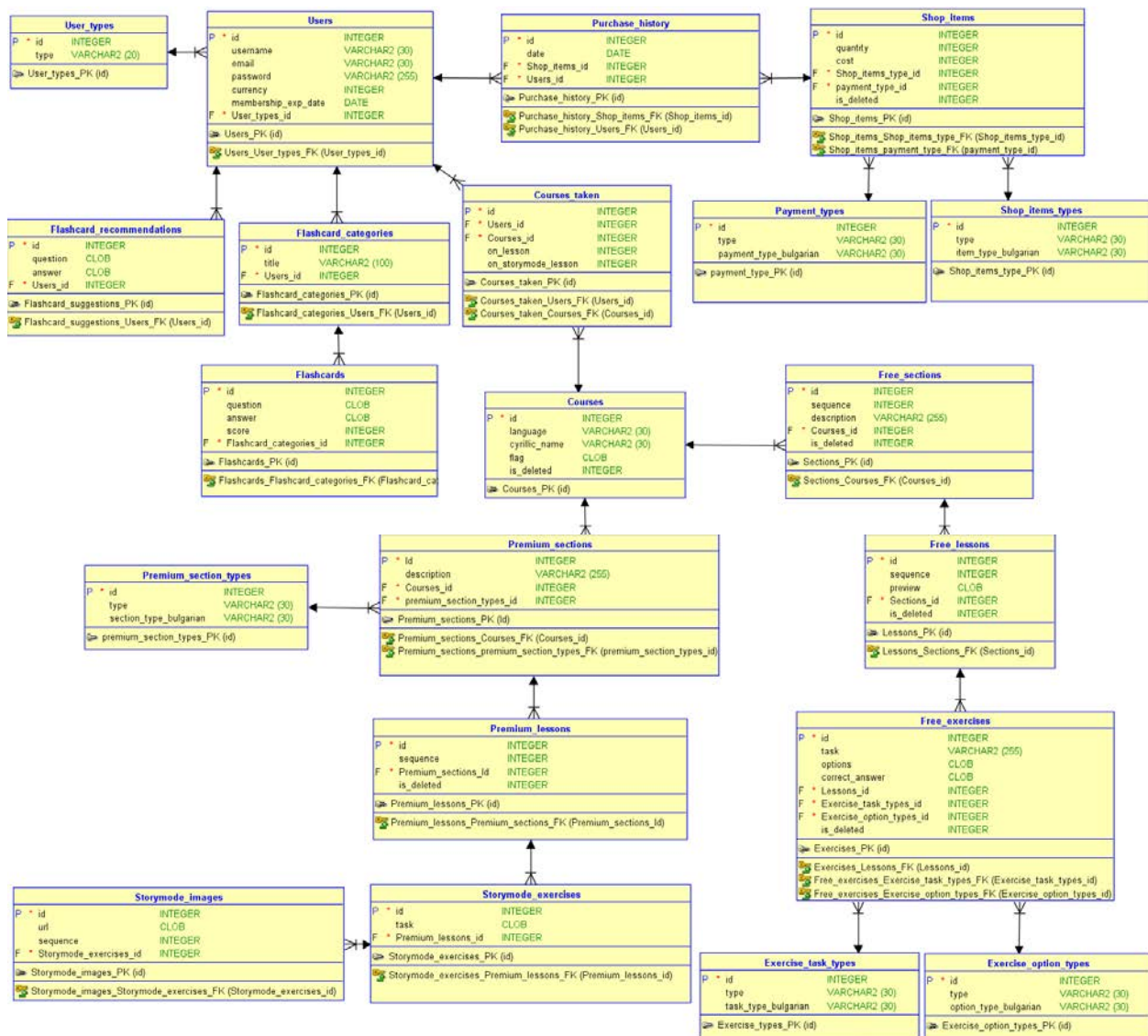
При използването на база от данни съществува риск от възникване на аномалии, които основно са четири вида:

- **Излишество** – когато една и съща информация (един и същи запис) се съхранява на различни места в базата данни;
- **Актуализация** – при наличие на излишество, ако се актуализира запис, то има вероятност промяната да не се отрази навсякъде;
- **Вмъкване при запис** – когато се добавя запис с поле, което съдържа информация, която в момента не е налична;
- **Изтриване на запис** – когато изтриването на един запис налага изтриването и на други свързани записи, което води до загуба на информация.

Избягването на тези аномалии се постига чрез т.н. нормализация на базата от данни. Това е процес на организиране на полетата в релационната база по начин, който отговаря на определени изисквания. Съществуват три нормални форми;

- **Първа нормална форма (ПНФ)** – за всяко пресичане на ред и колона съществува една единствена стойност и никога списък от стойности. Всяка клетка има само една стойност;
- **Втора нормална форма (ВНГ)** – изисква се всяка колона без ключ да бъде напълно зависима от целия първичен ключ, а не само от неговата част. Това са случаи, в които първичният ключ се състои от повече от една колона.
- **Трета нормална форма (ТНФ)** – изисква се не само всяка колона без ключ да зависи от целия първичен ключ, но и колоните без ключ да са независими една от друга.

В настоящата разработвана система, базата е в първа нормална форма. Релационната схема на базата от данни може да бъде представена чрез следната схема:



Фиг. 20 Реляционна база от данни

1.2. Описание на таблиците от базата от данни

Базата от данни се състои от 21 таблици, които ще бъдат описани по-подробно.

Таблица users:

В тази таблица се съхранява информацията за регистрираните потребители в системата.

Поле:

- id – INT - поле, представляващо уникален идентификатор на таблицата, наричан още първичен ключ (primary key). Служи за уникалност на

записите чрез автоматично инкрементиране на стойността от страна на MySQL.

- username – VARCHAR (30) - текстово поле за съхранение на име на потребител (обикновен потребител или администратор). Полето е ограничено до 30 символа.
- email – VARCHAR (30) - текстово поле за съхранение на е-майл на потребител.
- password – VARCHAR(255) - текстово поле за съхраняване на паролата на потребител.
- currency – INT – числова стойност за броя налични монети на потребител. Стойността по подразбиране е зададена на ,0‘.
- membership_exp_date – DATETIME – запазва стойност във формат 'yyyy-MM-dd HH:mm:ss' за краен срок на изтичане на абонамента на потребител. Стойността по подразбиране е зададена на NULL.
- user_types_id – INT – поле за вторичен ключ (foreign key). Използва се за връзка с таблица user_types, с което се посочва типа на потребител.

Таблица user_types:

В тази таблица се съхранява информация за типовете потребители. Предварително за записани възможните видове потребители – обикновен потребител (user), главен администратор (main_admin), администратор (admin).

id	type
1	admin
2	user
3	main_admin
NULL	NULL

Полета:

Фиг. 21 Табл. user_types

- id – INT – поле за първичен ключ (primary key).
- type – VARCHAR (20) – текстово поле за съхраняване на тип на потребител.

Таблица courses:

В тази таблица се съхранява информация за съществуващите курсове в приложението.

Полета:

- id – INT - поле за първичен ключ (primary key).
- language – VARCHAR (30) – текстово поле за съхраняване на името на добавен език, изписано на латиница.

- `cyrillicName` – VARCHAR (30) - текстово поле за съхраняване на името на добавен език, изписано на кирилица (български език).
- `Flag` – TEXT – текстово поле за съхранение на пътя до снимка на флаг за държавата, съответстваща на добавения език.

Таблица `courses_taken`:

В тази таблица се съхранява информация за курсовете, започнати от потребител.

Поleta:

- `id` – INT - поле за първичен ключ (primary key).
- `courses_id` – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица `courses`, с което се посочва конкретния курс, започнат от потребителя.
- `users_id` – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица `users`, с което се посочва потребителят, който е започнал даденият курс.
- `on_lesson` – INT – числова стойност за съхранение на урока, до който потребителят е достигнал от безплатните раздели. Стойността по подразбиране е ,1‘.
- `on_storymode_lesson` – INT - числова стойност за съхранение на урока, до който потребителят е достигнал от раздел „Истории“, който е част от платените раздели. Стойността по подразбиране е ,1‘.

Таблица `free_sections`:

В тази таблица се съхранява информация за безплатните раздели, добавени в приложението.

Поleta:

- `id` – INT - поле за първичен ключ (primary key).
- `sequence` – INT – поле за съхранение на числова стойност за поредност на раздел.
- `description` – VARCHAR (255) – текстово поле за съхранение на описание на раздел. Стойността по подразбиране е зададена на NULL.
- `courses_id` – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица `courses`, с което се посочва конкретният курс, към който принадлежи раздела.

- `is_deleted` – INT – числово поле, показващо дали конкретният раздел е активен или не. Стойността по подразбиране е ,0‘, отговарящо за активен раздел.

Таблица `free_lessons`:

В тази таблица се съхранява информация за безплатните уроци, добавени в приложението.

Поleta:

- `id` – INT - поле за първичен ключ (primary key).
- `preview` – TEXT – текстово поле за съхраняващо информация за урок
- `sequence` - INT – поле за съхранение на числова стойност за поредност на урок.
- `sections_id` – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица `free_sections`, с което се посочва конкретният раздел, към който принадлежи урока.
- `is_deleted` – INT – числово поле, показващо дали конкретният урок е активен или не. Стойността по подразбиране е ,0‘, отговарящо за активен урок.

Таблица `free_exercises`:

В тази таблица се съхранява информация за безплатните упражнения, добавени в приложението.

Поleta:

- `id` – INT - поле за първичен ключ (primary key).
- `task` – VARCHAR (255) – текстово поле за съхранение на заданието на упражнение.
- `options` – TEXT – текстово поле за съхранение на възможните отговори за упражнение.
- `correct_answer` – TEXT – текстово поле за съхранение на правилния отговор за упражнение.
- `lessons_id` – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица `free_lessons`, с което се посочва конкретният урок, към който принадлежи упражнението.
- `exercise_task_types_id` – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица `exercise_task_types`, с което се посочва типа на заданието на упражнение.

- exercise_option_types_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица exercise_option_types, с което се посочва типа на възможните отговори за упражнение.
- is_deleted - INT – числово поле, показващо дали конкретното упражнение е активно или не. Стойността по подразбиране е ,0‘, отговарящо за активно упражнение.

Таблица exercise_task_types:

В тази таблица се съхранява информация за типовете задание към дадено упражнение. Предварително за записани възможните видове задание както на английски, така и на български език.

id	type	task_type_bulgarian
1	listening	Слушане
2	reading	Четене
NULL	NULL	NULL

Фиг. 22 Табл exercise_task_types

Поleta:

- id – INT - поле за първичен ключ (primary key).
- type – VARCHAR (30) – текстово поле за съхранение на типа задание, изписано на английски език.
- task_type_bulgarian – VARCHAR (30) – текстово поле за съхранение на типа задание, изписано на български език.

Таблица exercise_option_types:

В тази таблица се съхранява информация за типовете възможни отговори към дадено упражнение. Предварително за записани видовете възможни отговори както на английски, така и на български език.

id	type	option_type_bulgarian
1	multiple_choice	Множествен отговор
2	make_sentence	Съставяне на изречение
3	text	Текст
NULL	NULL	NULL

Фиг. 23 Табл. exercise_option_types

Поleta:

- id – INT - поле за първичен ключ (primary key).
- type – VARCHAR(30) – текстово поле за съхранение на типа възможни отговори, изписано на английски език.
- option_type_bulgarian – VARCHAR(30) – текстово поле за съхранение на типа възможни отговори, изписано на български език.

Таблица **premium_sections**:

В тази таблица се съхранява информация за безплатните раздели, добавени в приложението.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- description – VARCHAR (255) – текстово поле за съхранение на описание на платен раздел. Стойността по подразбиране е зададена на NULL
- premium_section_types_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица premium_section_types, с което се посочва типа на платения урок.
- courses_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица courses, с което се посочва конкретният курс, към който принадлежи раздела.

Таблица **premium_section_types**:

В тази таблица се съхранява информация за типовете платени уроци. Предварително за записани възможните типове уроци, както на английски, така и на български език.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- type – VARCHAR (30) – текстово поле за съхранение на типа на възможните платени раздели, изписани на английски език.
- section_type_bulgarian – VARCHAR (30) – текстово поле за съхранение на типа на възможните платени раздели, изписани на български език.

Таблица **premium_lessons**:

В тази таблица се съхранява информация за платените уроци, добавени в приложението.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- sequence - INT – поле за съхранение на числова стойност за поредност на урок

- premium_sections_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица premium_sections, с което се посочва конкретният платен раздел, към който принадлежи урока.
- is_deleted – INT – числово поле, показващо дали конкретният урок е активен или не. Стойността по подразбиране е ,0‘, отговарящо за активен урок.

Таблица storymode_exercises:

В тази таблица се съхранява информация за платените упражнения от тип „Истории“, добавени в приложението.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- task – TEXT – текстово поле за съхранение на заданието на упражнение.
- premium_lessons_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица premium_lessons, с което се посочва конкретният урок, към който принадлежи упражнението.

Таблица storymode_images:

В тази таблица се съхранява информация за снимките към упражненията от тип „Истории“.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- url – TEXT – текстово поле за съхранение на пътя до снимка.
- sequence- INT – поле за съхранение на числова стойност за поредност на снимка.
- storymode_exercise_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица storymode_exercises, с което се посочва конкретният урок, към който принадлежи снимката.

Таблица flashcard_categories:

В тази таблица се съхранява информация за категории за флашкарти, които потребителите са създали.

Поleta:

- id – INT - поле за първичен ключ (primary key).

- title – VARCHAR (100) – текстово поле за съхранение на заглавие на категория за флашкарти.
- users_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица users, с което се посочва потребителят, който е създал категорията.

Таблица flashcards:

В тази таблица се съхранява информация за флашкарти, които потребителите са създали.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- question – TEXT – текстово поле за съхранение на въпрос за флашкарта.
- answer – TEXT - текстово поле за съхранение на отговор за флашкарта.
- score – INT – поле за съхранение на числова оценка на познаване на флашкарта.
- flashcard_categories_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица flashcard_categories, с което се посочва категорията, към която е добавена флашкартата.

Таблица flashcard_recommendations:

В тази таблица се съхранява информация за флашкарти, които да бъдат препоръчани на потребител.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- question – TEXT – текстово поле за съхранение на въпрос за флашкарта.
- answer - TEXT - текстово поле за съхранение на отговор за флашкарта.
- users_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица users, с което се посочва потребителят, на когото е препоръчана флашкартата.

Таблица shop_items:

В тази таблица се съхранява информация за продукти в магазина, които са добавени в приложението.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- quantity – INT – поле за съхранение на числова стойност за количество на продукта.
- cost – INT - поле за съхранение на числова стойност за цена на продукта.
- shop_item_types_id - INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица shop_item_types, с което се посочва типът на продукта.
- payment_types_id - INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица payment_types, с което се посочва типът на необходимото плащане за закупуване на продукта.
- is_deleted – INT – числово поле, показващо дали конкретният урок е активен или не. Стойността по подразбиране е ,0', отговарящо за активен урок.

Таблица purchase_history:

В тази таблица се съхранява информация за историята на покупките, които даден потребител е извършил.

Поleta:

- id – INT - поле за първичен ключ (primary key).
- date – DATETIME – запазва стойност във формат 'yyyy-MM-dd HH:mm:ss' за текуща дата на извършване на поръчката.
- shop_items_id - INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица shop_items, с което се посочва продуктът, който е бил закупен.
- users_id – INT - поле за вторичен ключ (foreign key). Използва се за връзка с таблица users, с което се посочва потребителят, извършил покупката.

Таблица shop_item_types:

В тази таблица се съхранява информация за типовете продукти в магазина. Предварително за записани възможните видове продукти както на английски, така и на български език.

id	item_type	item_type_bulgarian
1	coins	Монети
2	membership	Абонамент
NULL	NULL	NULL

Фиг. 24 Табл. shop_item_types

Полега:

- id – INT - поле за първичен ключ (primary key).
- item_type – VARCHAR (30) – текстово поле за съхранение на типа на продукт, изписано на английски език.
- item_type_bulgarian – VARCHAR (30) – текстово поле за съхранение на типа на продукт, изписано на български език.

Таблица payment_types:

В тази таблица се съхранява информация за типовете плащания чрез магазина. Предварително за записани възможните видове плащания както на английски, така и на български език.

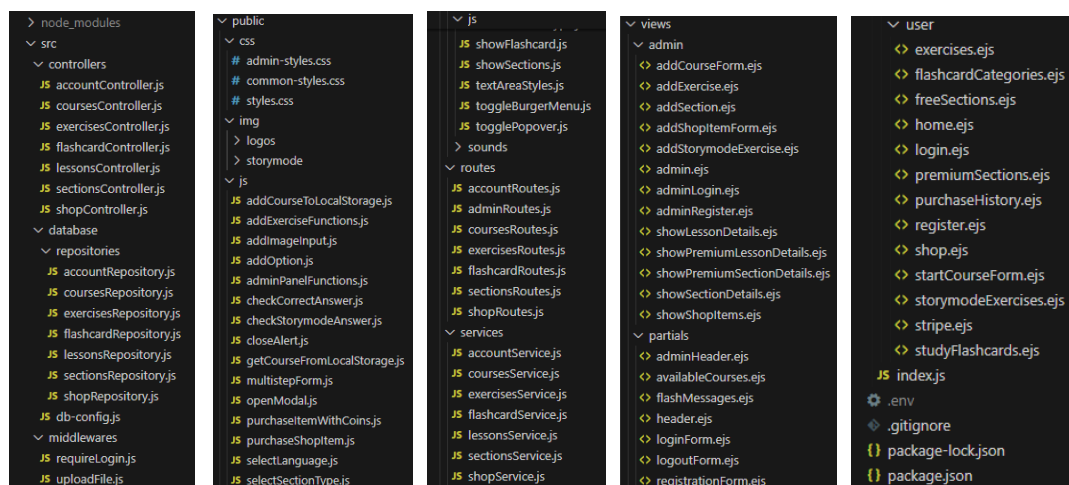
id	payment_type	payment_type_bulgarian
1	app_currency	Монети
2	money	Пари
NULL	NULL	NULL

Фиг. 25 Табл payment_types

Полега:

- id – INT - поле за първичен ключ (primary key).
- payment_type – VARCHAR (30) – текстово поле за съхранение на типа на плащане, изписано на английски език.
- payment_type_bulgarian – VARCHAR (30) – текстово поле за съхранение на типа на плащане, изписано на български език.

2. Описание на програмните модули



Фиг. 26 Структура на проекта

Приложението е съставено от следните програмни модули:

- Routes (пътища) – тук се описват всички endpoint-и и типа на заявките (get, post, patch, delete);
- Controllers (контролери) – изпратената заявка през routes извиква функция в контролерите, която служи за обработка на данните. Данните могат да бъдат изпратени през параметрите на url-a, през body на post, patch или delete заявки или през сесията. Тези данни след това се изпращат към сървисите, а след получаване на отговор от тях се прави връзка с изгледите и се визуализира извиканата страница със съдържанието, което е получено;
 - o accountController
 - createUser – функция за създаване на потребител. Входните данни се изпращат към сървиса и полученият отговор се обработва. Ако е изникнала грешка, се генерира съобщения за грешка, известяващо потребителите каква е възникналата грешка. Ако регистрацията е била успешна, данните за user_id, user_type и user_courses се запазват в сесията и се пренасочва към страницата за уроците;
 - loginUser – функция за вход на потребител. Ако входът е успешен, данните на потребителя се записват в сесията;
 - logoutUser – функция за изход от профил. Разрушава се сесията с всички записани данни за потребителя.
 - o coursesController
 - o exercisesController
 - o flashcardController
 - o lessonsController
 - o sectionsController
 - o shopController
 - makePayment – създава се Stripe payment intent за извършване на тестова покупка с банкова карта през Stripe. При успешно извършена покупка, Stripe връща отговор за успех, след което се извикват сървисите за обработване и предоставяне на закупения продукт.
- Services (сървиси) – тук се описва цялата бизнес логика на приложението. Прави се връзка със слоя на базата данни, където се

изпращат получените данни от контролерите, а след получаване на отговор от базата, той се изпраща обратно към контролера;

О accountService

- createUser – съдържа бизнес логиката за добавяне на потребител. Първо се проверява дали въведеното потребителско име е заето, след което правят проверки за входните данни от страна на потребителя – потребителското име и парола, с regex. Ако данните са коректни, паролата се хешира и потребителят се записва в базата данни;
- loginUser – при изпращане на заявка за вход в системата, първо се проверява съществува ли потребител с такова потребителско име. Ако съществува, се вземат данните за потребителя заедно с хешираната парола, която се декриптира и се сравнява с паролата въведена от потребителя;
- updateUserDataForCompletedLesson – при завършване на безплатен урок, прогресът на потребителя за безплатните уроци се актуализира и му се предоставя награда под формата на монети;
- updateUserDataForCompletedPremiumLesson - при завършване на платен урок, прогресът на потребителя за платените уроци се актуализира и му се предоставя награда под формата на монети единствено при първо преминаване на урок;
- updateMembership – при за купуване на абонамент, крайният срок за изтичане се актуализира, в случай че вече няма наличен абонамент. Извършва се форматиране на дати, за да се запазят и визуализират в лесно четим за потребителя формат. Ако абонаментът е бил закупен с монети, се актуализира броя монети на потребителя;
- checkIfUserHasMembership – проверка дали потребителят притежава абонамент. Използва се, за да се определи дали има достъп до платените раздели

О coursesService

- addCourse – добавяне на нов курс от страна на администраторите, в случай че вече няма добавен такъв курс;

- addCourseForUser – добавяне на нов курс от страна на потребител. Прави се проверка дали потребителят вече има добавен даденият курс, с цел по-добра защита при опит за добавяне на курс извън програмния интерфейс.
- o exercisesService
 - addExercise – добавяне на безплатно упражнение към урок. Извършват се проверки за въведените данни за „възможни отговори“ и „правилен отговор“. Ако в променливата с възможните отговори се съдържат празни полета, те се изчистват, след което според типа на отговора се прави проверка дали е възможно достигането до правилния отговор чрез въведените възможни отговори. Ако това условие не е изпълнено, упражнението не се добавя в базата данни;
 - updateFreeExercise – редактиране на безплатно упражнение. Подобно с функцията за добавяне на упражнение и тук се извършват проверките за коректно въведени данни за „възможни отговори“ и „правилен отговор“;
 - checkUserAnswer – при отговаряне на упражнение от страна на потребителя се изпраща заявка към backend логиката, където се извършва проверка дали отговорът е правилен.
- o flashcardsService
- o lessonsService
 - addLesson – добавяне на безплатен урок. Проверява се последователността на последно въведеният урок и се добавя новият с една единица отгоре;
 - addPremiumLesson – добавяне на платен урок. Проверява се последователността на последно въведеният урок и се добавя новият с една единица отгоре.
- o sectionsService
 - addSection – функция за добавяне на безплатни и платени раздели. За безплатните раздели се проверява последователността на последния добавен раздел и се добавя новият с една единица отгоре;
 - deleteFreeSection – изтриване (деактивиране) на раздел. Условието за изтриване на раздел е той да няма налични

уроци, за да не се наруши последователността на уроците и прогреса на потребителите.

- o shopService
- Database (база от данни)
 - o db-config.js – файл за създаване на връзка към базата данни;
 - o repositories – директория, в която се съхраняват всички файлове, съдържащи заявките към базата данни. С получените данни от сървисите се изпълняват нужните заявки към базата, след което данните от отговора на заявките се връща обратно към сървисите, където може да се приложи допълнителна бизнес логика с получените данни;
 - accountRepositories
 - coursesRepositories
 - exercisesRepositories
 - flashcardRepositories
 - lessonsRepositories
 - updateFreeLessonDeleteStatus – при редактиране на статуса за видимост на безплатен урок, трябва да се гарантира, че останалите компоненти в базата данни са също актуализирани. За да се запази коректността на данните, се използва SQL транзакция, която гарантира, че всички таблици ще бъдат редактирани успешно;
 - updateStorymodeLessonDeleteStatus – редактиране на статуса за видимост на платен урок. Използва се SQL транзакция.
 - sectionsRepositories
 - shopRepositories
- Views (изгледи) – създават се всички изгледи, с които потребители си взаимодействат. Визуализират се данните, получени от контролерите;
 - o user – директория съдържаща всички потребителски изгледи;
 - o admin – директория, съдържаща всички администраторски изгледи;
 - o partials – директория, съдържаща компоненти, които се използва многократно в останалите изгледи (напр. хедъри, форми за вход и регистрация и др.)
- Middlewares (мидълуеъри) – междинен слой, служещ за извършване на допълни проверки при изпращане на заявки към слоя на пътищата. В

конкретното задание се използва за проверка дали потребителят е записан в сесията и ако е - какъв е типът на профила му (обикновен потребител или администратор);

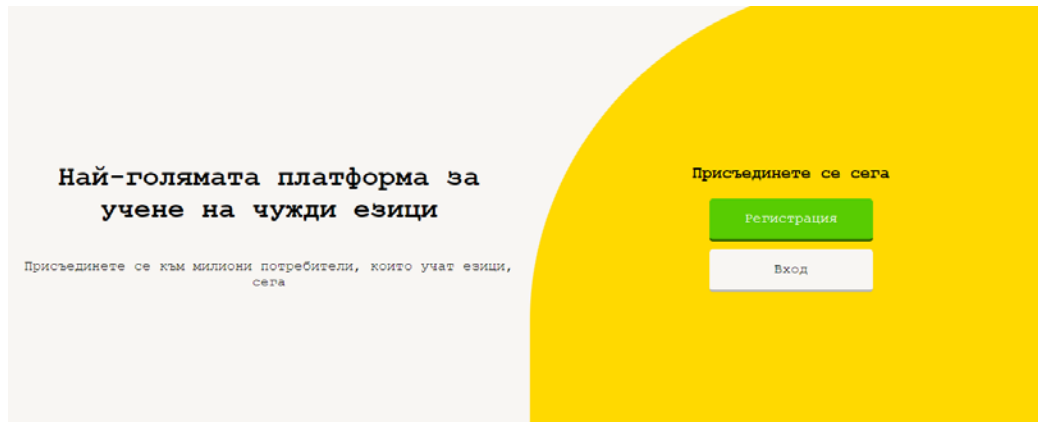
- o requireLogin – функции за проверка на тип на вписаният потребител и запазена ли е стойност за id на потребител в сесия. Използват се за защита на достъпа до endpoint-ите.
- o uploadFile – функция за записване на файл в дадена директория. Използва се за платените уроците от тип „истории“.
- Public – директория, съдържаща всички css файлове, снимки, аудио файлове и javascript файлове, които се използват от frontend частта. Javascript файловете са изключително важни за предоставяне на динамично взаимодействие между потребител и интерфейс. Някои от най-важните файлове в тази директория:
 - o checkCorerctAnswer – служи за извеждане и проверка на безплатни упражнения. В този файл се създава и дизайна на наличните упражнения, изпраща се заявка за получаване на данните от backend логиката, получава се отговора на потребителя и се изпраща за проверка отново в backend логиката;
 - o checkStorymodeAnswer – извеждане и проверка на платени раздели от тип „истории“. Създава се дизайна на упражнението;
 - o openFlashcardModals – създаване на модални прозорци за страницата с флашкарти. Чрез тях потребителите добавят, редактират и изтриват флашкартите си;
 - o multistepForm – създава формата за регистрация за потребители, разделена на две стъпки;
 - o addOption / imageInput – динамично добавяне на полета за въвеждане на „възможни отговори“ за безплатни и платени раздели.

3. Структура и организация на потребителския интерфейс

3.1. Потребителска страница

При първоначално влизане в системата, за потребителя се визуализира началният екран, от който може да се избере дали да се направи регистрация или да се влезе в системата, ако потребителят вече има профил, в който не е влязъл. Ако

потребителят не е влязъл в профила, си няма достъп до останалата част от системата, докато не се впише успешно в нея.



Фиг. 27 Начален екран на приложението

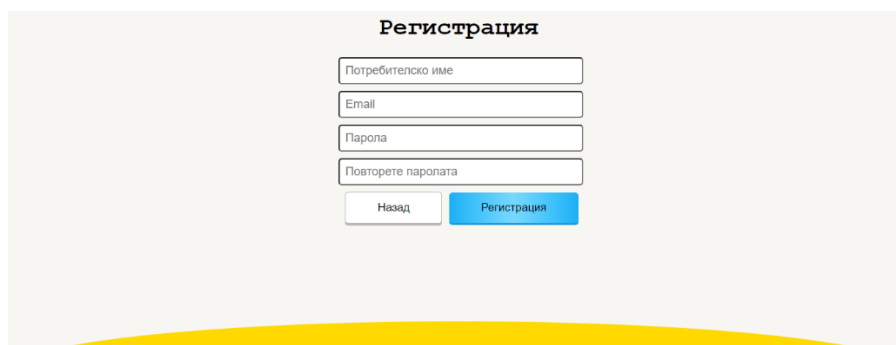
При избор на регистрация се визуализира форма за регистрация, която е от две части:

- Първо потребителят трябва да избере езика, който иска да започне да изучава:



Фиг. 28 Избор на курс при регистрация

- След това потребителят трябва да въведе своите данни за регистрация:

Изображение на екрана за въвеждане на данни за регистрация. Върху е заглавието "Регистрация". Има четири текстови полета: "Потребителско име", "Email", "Парола" и "Повторете паролата". В долната част са разположени бутоните "Назад" и "Регистрация".

Фиг. 29 Въвеждане на данни за регистрация

Данните, въведени от потребителя, трябва да отговарят на следните изисквания, за да са валидни:

- Потребителското име не може да съдържа букви, различни от латинските;
- Паролата трябва да бъде минимум 8 символа. Не е избрана по-силна защита за паролите, тъй като не се предполага сериозно изтичане на данни от приложението, а измислянето на по-сложна парола може единствено да откаже потребителите от това да си създадат профил;
- Полетата за потребителските данни да не бъдат празни.

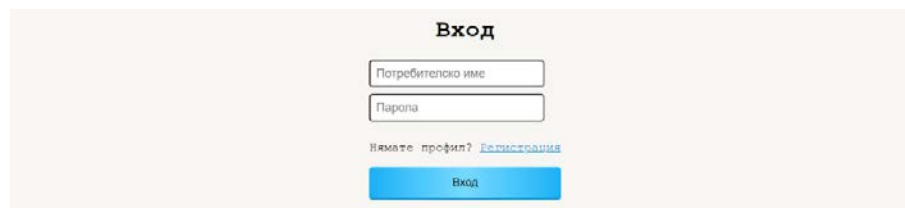
Преди записване на данните на потребителите в базата данни, паролите се хешират с цел по-добра защита. Криптографският алгоритъм, използван в приложението е Vcrypt, който е широко използван поради голямата си ефективност срещу brute-force атаки и способността му да се адаптира към нарастващата изчислителна мощност. Освен включването на „сол“ за защита срещу атаки, с течение на времето броят на итерациите, с които се криптират данните, може да се увеличи, за да стане криптирането по-бавно. По-големият брой итерации прави хеширането по-силно, което може все повече да се увеличава с подобряването на устройствата на потребителите. Към днешно време стандартът за брой итерации е 12, което отговаря на средностатистическата мощност на използваните компютри. Тази е и стойността, която е избрана да бъде използвана в проекта. Алгоритъмът също така прибавя „сол“ към криптираната парола, която е различна всеки път и това допълнително увеличава сигурността, тъй като гарантира, че дори еднакви пароли не са хеширани по един и същи начин.

Ако въведените данни не са коректни, се извежда съобщение за грешка и потребителят трябва отново да повтори регистрацията. При коректно въведени данни потребителят бива вписан автоматично в системата и може веднага да започне изучаването на избрания от него език или да разгледа някои от останалите функционалности.

Ако потребителят има вече създаден профил, може да избере да се впише в системата чрез бутона за вход. Входът се извършва с данните за потребителско име и парола на потребителя, с които той се е регистрирал в системата.

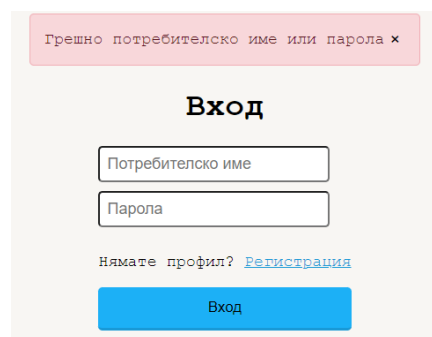
При получаване на данните за вход, първо се проверява дали съществува такъв потребител в базата данни по въведеното потребителско име. Единствено ако съществува, паролата се декриптира и сравнява с въведената от формата за вход. Vcrypt библиотеката предоставя една обща функция, която извършва

декриптирането и сравняването. Ако и двете условия са изпълнени, функцията връща данните на потребителя и те биват записани в сесията. С най-голяма важност е типа на потребителя, който желае да влезе в системата. Обикновеният потребител може да влезе с данните си за вход единствено през формата за вход за потребители. В случай, че пробва да влезе през формата за вход за администратори, то той ще види грешка за грешно въведени данни за вход. Чрез тази стойност също се гарантира и че неадминистраторски профил няма да получи достъп до правата на администраторите.



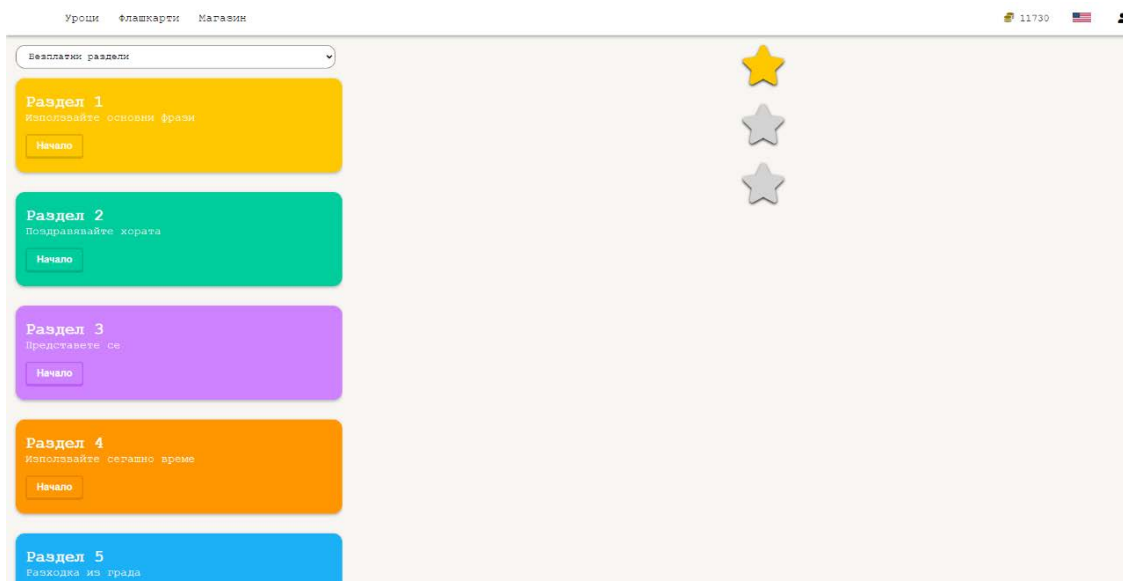
Фиг. 30 Форма за вход

Ако данните за потребителско име и парола са грешни, отново се визуализира съобщение за грешка, като не се предоставя точна информация кое поле от формата за вход е сгрешено. Това се прави с цел допълнителна защита при опит за влизане в чужд профил, за да не знае хакерът дали е въвел правилно данните в едно от двете полета.



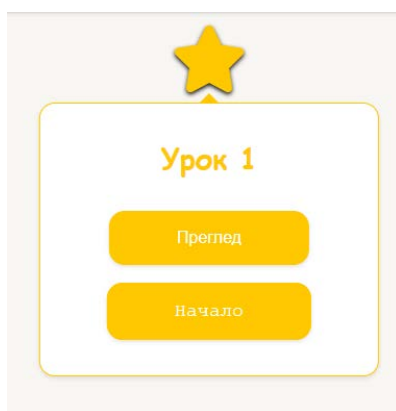
Фиг. 31 Съобщение за грешка при вход

Ако потребителят се е вписал успешно, получава достъп до началния екран за наличните уроци за езика, който изучава.



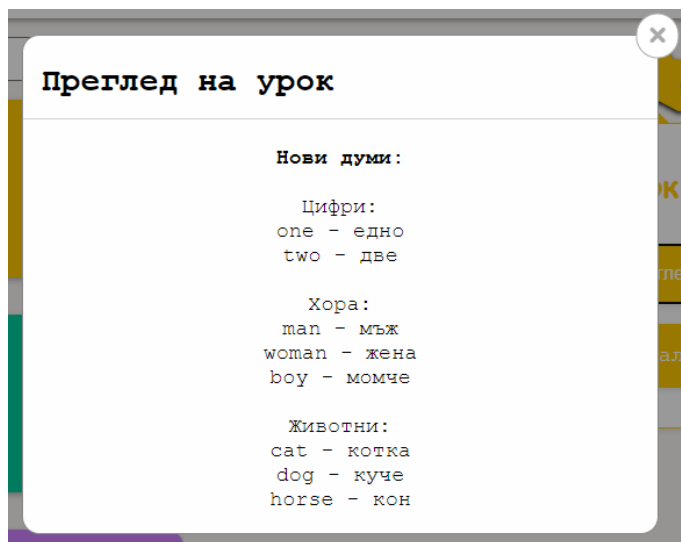
Фиг. 32 Екран за преглед на безплатни раздели и уроци

Потребителите имат достъп до уроците, до които са прогресирали. Тези, които са достъпни, са в жълт цвят, а останалите в сив. При избор на урок се отваря изскачащ прозорец, от който може да се започне избраният урок.



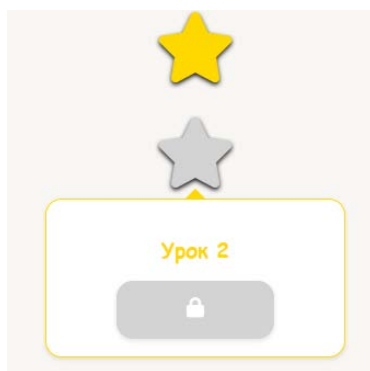
Фиг. 33 Изскачащ прозорец за начало на урок

С бутона за преглед потребителите могат да се запознаят с основната информация, която ще научат в урока, в случай, че такава е въведена от администраторите. В този прозорец администраторите ще могат да въвеждат всичко нужно, което потребителят трябва да знае, като нови думи, граматични правила и тн. Ако не е въведена информация за преглед на урока, този бутон не се визуализира.



Фиг. 34 Преглед на урок

За невалидните уроци бутонът за начало е в сив цвят с икона на катинар:



Фиг. 35 Изскачащ прозорец за начало на урок

Ако урокът е достъпен, се отваря страницата с упражненията. Разработени са няколко типа упражнения от безплатните уроци.

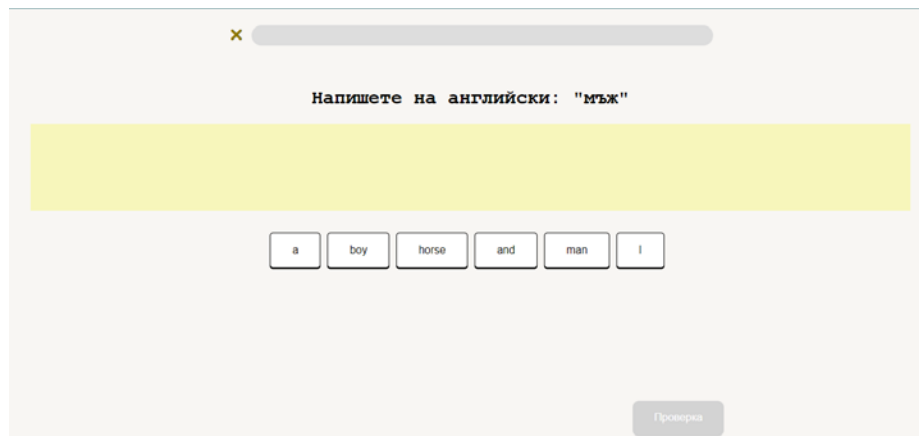
По задание те могат да бъдат:

- За четене;
- За слушане.

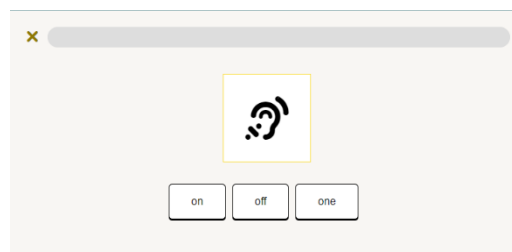
По варианти за отговор те могат да бъдат:

- С множествен отговор – избира се един от възможните отговори, които са предоставени;
- Съставяне на изречение – от предоставените думи се съставя изречение.

Опциите за задание и отговори могат да бъдат комбинирани по всякакъв начин. При започване на урока се използва алгоритъм за разбъркване на всички въпроси, както и на възможните отговори, които са предоставени. Алгоритъмът приема масив като параметър и извършва цикъл, като започва от последния елемент в масива, стигайки до първия. При всяка итерация се избира случаен индекс j и се разменя стойността с текущия индекс i , на който се намира цикълът. По този начин се гарантира, че всеки път упражненията са разбъркани, както и техните отговори, което би могло да предотврати потребителят да запомни упражнението по позицията, в която се намира упражнението, или по позицията на възможните отговори.

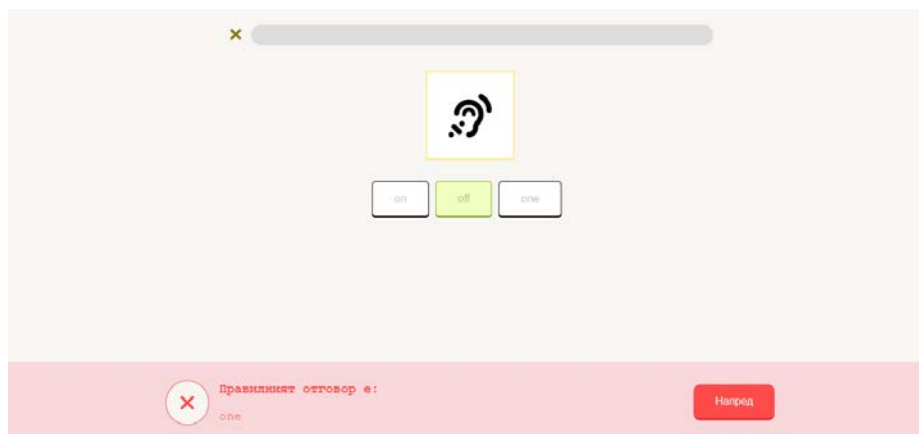


Фиг. 36 Урок от тип четене и съставяне на изречение



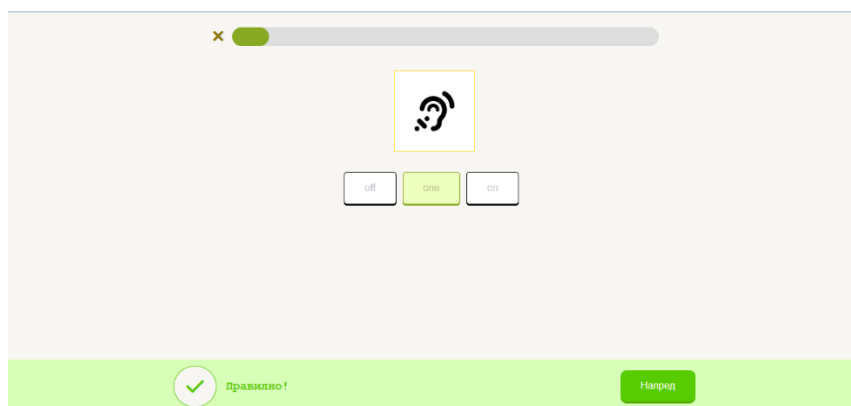
Фиг. 37 Урок от тип слушане и множествен отговор

При грешен отговор се извежда съобщение за грешка и се изписва правилният отговор, за да може потребителят да го запомни. Също така въпросът и отговорът автоматично се добавят в прозореца за препоръчани флашкарти в страницата за преглед на флашкарти. Ако потребителят направи грешката отново, но упражнението вече е било записано, тогава не се прави повторно записване в таблицата за препоръчани флашкарти в базата.



Фиг. 38 Грешен отговор

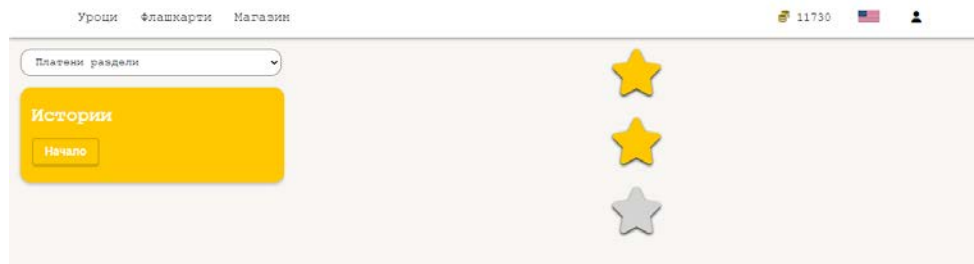
Ако потребителят е отговорил правилно, се извежда съобщение за успех и той може да премине успешно към следващото упражнение.



Фиг. 39 Правилен отговор

При успешно преминаване на всички упражнения, на потребителя се предоставя съответната награда под формата на монети, които може да използва в магазина на приложението, и следващият урок става достъпен, ако все още не е бил. Прави се актуализация на записаните данни в сесията, за да бъдат видими за потребителя без да се налага тя да се създава отново.

При наличие на закупен абонамент потребителите получават достъп до платените раздели. Платените раздели могат да бъдат достъпни чрез падащото меню, намиращо се над секцията с разделите. Потребителите без абонамент също могат да разгледат наличните платени раздели, но вместо иконите с уроците за тях ще се изпише текст, уведомяващ ги, че нямат закупен абонамент. За момента е проектиран един платен раздел:



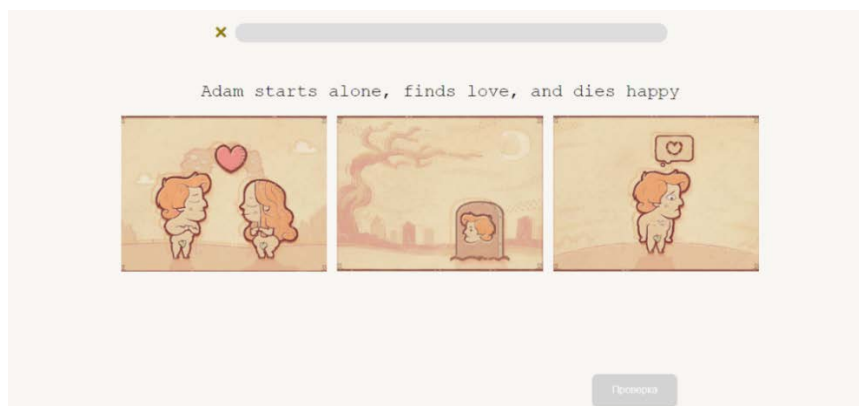
Фиг. 40 Преглед на платени раздели

Прогресът на платените уроци се запазва отделно от този на безплатните уроци. Тоест преминаването на платените уроци не влияе и не е свързан с безплатните.

Платените уроци от тип „Истории“ предоставят на потребителите възможност допълнително да развият своите умения за четене с разбиране. Предоставя се кратка история с няколко на брой снимки, които потребителите трябва да свържат, за да получат правилната последователност на описаната история.

При този раздел също се използва алгоритъм за разбъркване на масива, но единствено на частта с отговорите за упражнението. Заданията следват последователността, с която са въведени в базата данни.

Потребителите трябва да маркират снимките на екрана чрез натискане на изображението, което ги номерира и тази номерация се сравнява с последователността на снимките, записана в базата. Ако се демаркира някоя снимка, то последователността на останалите автоматично се актуализира, за да не се налага потребителите да демаркират всички снимки, за да започнат отначало. Например, ако са предоставени 3 изображения, които са били маркирани, и потребителят демаркира тази с последователност 2, то тази с последователност 1 ще запази своята стойност, но следващата със стойност 3 ще се актуализира в 2.



Фиг. 41 Упражнение на платен раздел „Истории“ [\[9\]](#)

Ако свържат правилно картинките от историята, се извежда съобщение за успех и могат да продължат към следващото упражнение. При грешка трябва отново да повторят същото упражнение, като в тази категория не се показва правилният отговор на потребителите.



Фиг. 42 Правилен отговор

След успешно преминаване на урок, следващият става активен, ако все още не е бил. В този раздел потребителите могат да спечелят монети от урок единствено при първото му преминаване. При всяко следващо минаване на същия урок, потребителите не получават монети.

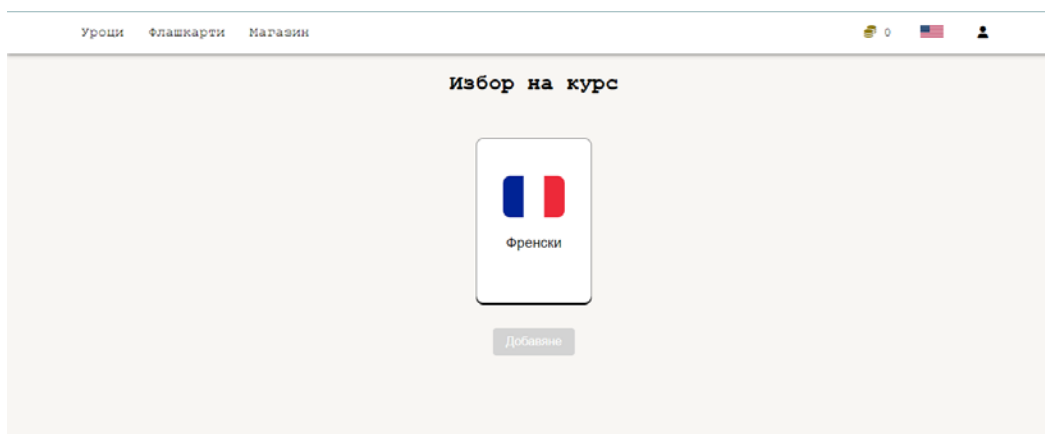
Освен визуално предоставяне или отказване на достъп до урок или раздел, се извършват и допълнителни проверки в backend логиката, с цел допълнителна защита при опит да се достъпи урок през url-а или Postman.

Потребители могат да започнат нов курс от добавените в приложението или да сменят на друг, който вече са започнали да изучават. Това може да се случи през header лентата, приплъзвайки мишката върху иконката на знамето на текущия избран език:



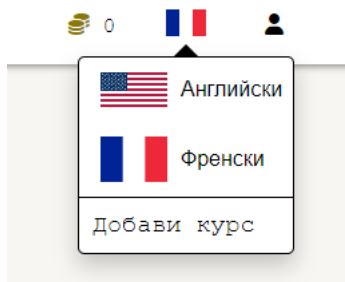
Фиг. 43 Преглед на падащо меню за курсове

При натискане на бутона „Добави курс“, потребителите биват насочвани към страницата за добавяне на нов курс. Тук се визуализират всички курсове, които са били добавени в системата от администраторите, но които не са били все още добавени за изучаване от потребителите.



Фиг. 44 Страница за стартиране на нов курс

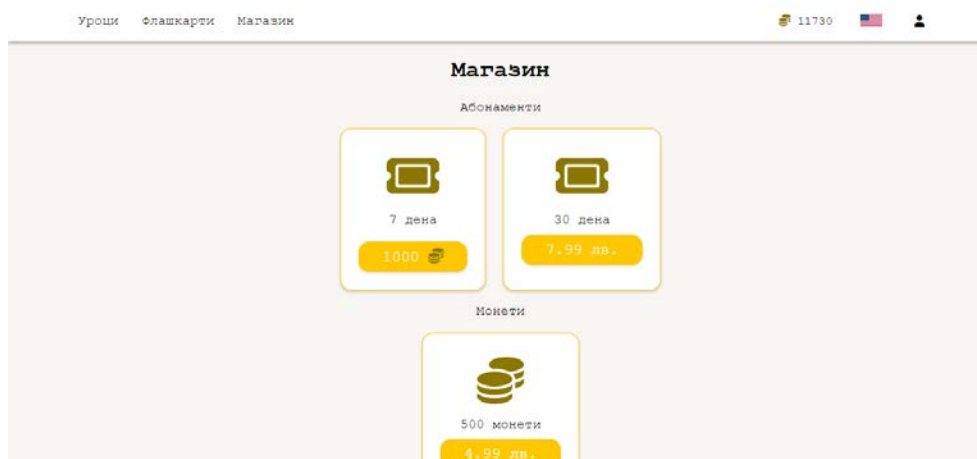
След като е избран желаният курс, потребителите могат да започнат изучаването на съответния език. Програмата ги пренасочва автоматично към уроците от новоизбраният курс. В падащото меню за започнатите курсове вече може да се види и току що добавеният курс:



Фиг. 45 Падащо меню за започнати курсове

Подобно на достъпването на уроци чрез други методи, различни от бутоните в приложението, се извършват проверки и дали курсът, който потребителят желае да достъпи, е добавен от него. Ако не е, то той бива пренасочен към страницата за започване на нов курс.

Потребители могат да използват спечелените си монети в магазина на приложението. Там те могат да закупят абонамент за една седмица напълно безплатно, единствено чрез преминаване на уроци. Също така могат да извършат покупка с банкова карта, като имат възможността да закупят абонамент и монети с реални пари.



Фиг. 46 Магазин

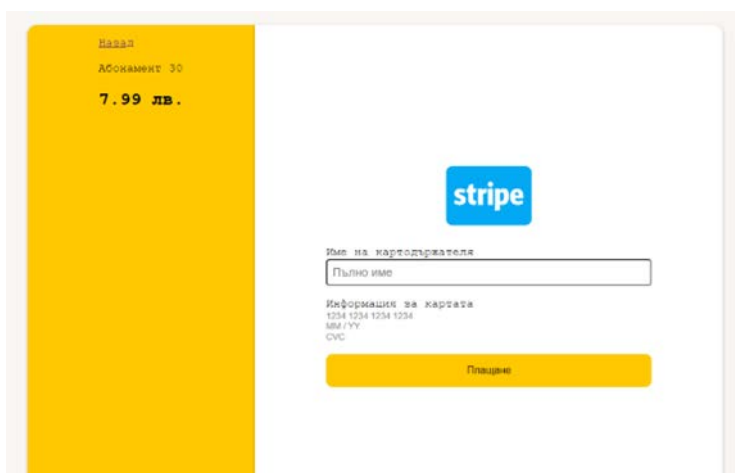
За да се закупи абонамент, потребителят трябва да не притежава активен в текущия момент абонамент. Ако това условие е изпълнено, на екрана се визуализира съобщение за успешно закупуване на абонамент. В противен случай потребителят вижда съобщение за грешка, което съобщава, че той вече има активен абонамент.

Данните за абонаментите се записват като дати, като тези дати предварително се форматира във вид „уууу-ММ-dd HH:mm:ss“, където “уууу” е година, “”ММ” –

месец, “dd” – ден, а “HH:mm:ss” са „час:минути:секунди“. При опит за достъпване на платен урок, датата за краен срок за изтичане на абонамента се проверява с текущата дата и се предоставя или отказва достъп до платеното съдържание.

При извършване на покупка с монети, единственото условие е потребителят да притежава нужната сума в профила си. Първо се проверява за наличността, а след това се извършва покупката, ако сумата е налична.

При извършване на покупка с банкова карта, потребителят се пренасочва към страница за плащане, където трябва да въведе своите данни за банковата си карта.

The image shows a payment interface. On the left, a yellow vertical bar contains the text "Назал", "Абонамент 30", and "7.99 лв.". To the right, there is a white form area. At the top of the form is the Stripe logo. Below it, there is a label "Име на картодръателя" followed by a text input field containing "Пълно име". Underneath that is a label "Информация за картата" followed by three input fields: "1234 1234 1234 1234", "MM / YY", and "CVC". At the bottom of the form is a yellow button labeled "Плащане".

Фиг. 47 Форма за плащане с банкова карта

При успешно извършване на покупката, приложението пренасочва потребителя обратно в страницата на магазина и се извежда съобщение, че покупката е била успешна. Ако потребителят въведе грешни данни за карта, се извеждат съответните съобщения, информиращи го, че данните не са коректни. Ако данните са били коректни, но се е получила грешка при обработването на покупката, потребителят бива пренасочен обратно в страницата на магазина, извежда се съобщение за неуспешно извършена покупка и той не бива таксуван.

Потребителите могат да преглеждат историята на извършените от тях покупки, приплъзвайки мишката през иконата за профила и натискайки бутона за отваряне на страницата за преглед на извършените покупки.

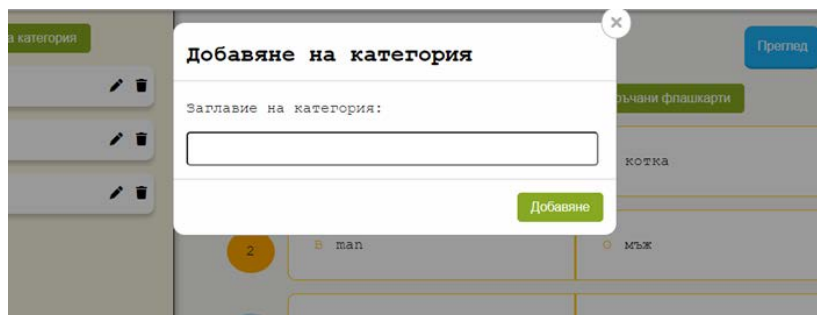
Уроци		Флашкарти	Магазин	11730			
Дата		Количество	Цена	Продукт			
2024-06-10 01:31:24		7 дни	1000	Абонамент			
2024-06-10 01:30:23		500 монети	4.99 лв.	Монети			
2024-06-10 01:35:56		500 монети	4.99 лв.	Монети			
2024-06-20 13:02:44		500 монети	4.99 лв.	Монети			
2024-06-10 01:47:22		30 дни	7.99 лв.	Абонамент			

Фиг. 48 Преглед на извършени покупки

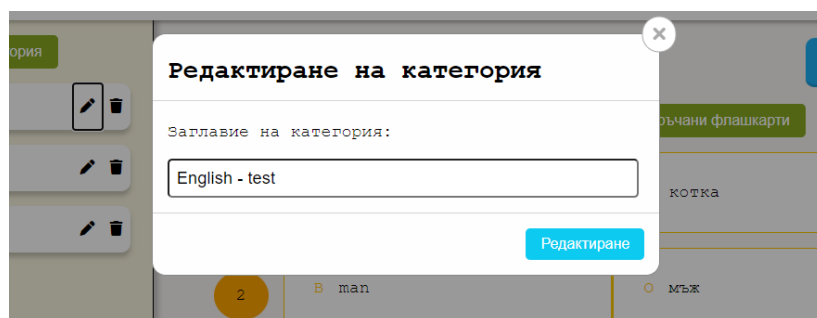
Освен преминаването на уроците, се предоставя и още един метод за обучение – чрез флашкарти. Всеки потребител напълно безплатно може да създава свои флашкарти, чрез които да преговаря наученото от уроците.

Фиг. 49 Страница за преглед на флашкарти

В тази страница потребителите могат да създават категории за флашкарти, където да групират по удобен за тях начин флашкартите, които ще създават. Също така могат да редактират или изтриват вече съществуващите категории и флашкарти. Добавянето, редактирането и изтриването се случва чрез модални прозорци, където потребителите могат да въведат нужните данни или да потвърдят изтриването.

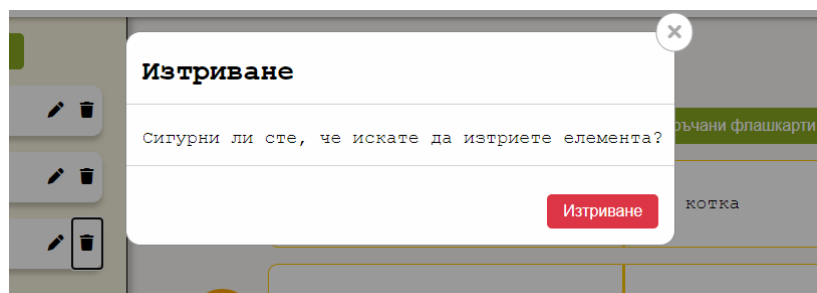


Фиг. 50 Добавяне на категория



Фиг. 51 Редактиране на категория

Изтриването на категории и флашкарти е перманентно и не може да бъде възвърнато дори от администраторите. Поради тази причина се извежда модален прозорец за потвърждение на избора на потребителя.



Фиг. 52 Изтриване на категория

При добавяне на нови флашкарти, те биват добавени към категорията, която е отворена от потребителя.

Фиг. 53 Добавяне на флашкарта

Чрез бутона за преглед на препоръчани флашкарти, потребителите могат да преглеждат грешките, които са допуснали при преминаване на безплатните уроци. Така те могат автоматично да добавят тези грешки като флашкарти, за да ги изучават допълнително. Упражнение за слушане не се запазват като препоръчани флашкарти, дори да са били сгрешени. Ако преценят, че вече не е нужно допълнително изучаване на тези грешки, могат да премахнат препоръчаната флашкарта и тя няма да бъде добавена към тяхната категория. За по-голямо удобно за потребителя, се визуализират първоначално първите 5 флашкарти, след което предоставения бутон, могат да се визуализират следващите 5.

Въпрос	Отговор	Статус
Напишете на английски: "мъж и момче"	a man and a boy	✓ X
Кое от посочените е "мъжът"	the man	✓ X
Напишете на английски: "мъж"	a man	✓ X
Изберете правилното значение: "and"	и	✓ X
Кое от посочените е "едно"	one	✓ X

Повече

Фиг. 54 Препоръчани флашкарти

След като добавят желаните от тях флашкарти, потребителите могат да започнат да ги изучават чрез бутона за „Преглед“.



Фиг. 55 Преглед на флашкарта

В тази страница освен да преглеждат флашкартите, те могат да оценят до колко добре познават всяка флашкарта. Чрез тази оценка след това те имат възможността да преглеждат само тези карти, които са с по-ниска оценка в категорията. Това спестява нуждата да изтриват останалите, ако все пак не желаят да го правят, но не желаят и да ги преглеждат.

При преместване на курсора на мишката върху флашкартата, автоматично се обръща на другата страна, за да се види отговора, записан на нея.

Чрез бутона „Ниски оценки“ могат да се преглеждат флашкартите, които са били ниско оценени. Визуализира се страница, която изглежда по същия начин като тази за преглед на всички флашкарти, но само с тези с ниска оценка.

3.2. Администраторска страница

Администраторите могат да влязат в своя профил чрез формата за вход за администратори. Всеки един endpoint за администраторския панел е защитен от това да не може да се достъпва от обикновени потребители. Това се случва чрез мидълуеъра, където се прави проверка за типа на потребителя, записан в сесията при успешен вход.

Ако входът е бил успешен, администраторът получава достъп до началния екран от панела, където се визуализират добавените курсове.

Всички изтривания в администраторския панел не са перманентни. В таблиците в базата данни се съхранява стойност `is_deleted`. Ако тя е `,0'`, елементът е

активен и може да се вижда от обикновените потребители и администраторите. Ако е ,1', не може да бъде виждан, но може да се възстанови от базата данни.

Администратор	Раздели
Курсове	Английски >
Добавяне на администраторски профил	Френски >
Добавяне на курс	
Добавяне на предмет в магазина	
Изход	

Фиг. 56 Преглед на курсове

От тази страница могат да се прегледат разделите, добавени в курсовете, както и да се добавят нови раздели към тях.

Английски ▼

Безплатни раздели

Раздел 1 - Използвайте основни фрази

Раздел 2 - Поздравявайте хората

Раздел 3 - Представете се

Раздел 4 - Използвайте сегашно време

Раздел 5 - Разходка из града

Раздел 6 - Членове на семейството

Платени раздели

Истории

Добавяне на раздел

Изтриване на раздел

Френски >

Фиг. 57 Преглед на раздели

От тук могат да се добавят нови раздели, чрез бутона за добавяне, който отваря нов прозорец с формата за добавяне на раздел.

Тип на раздела

Безплатен раздел ▼

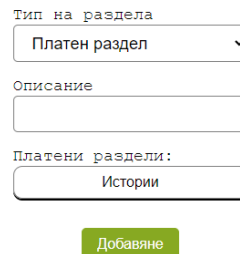
Описание

Добавяне

Фиг. 58 Форма за добавяне на безплатен раздел

При смяна на стойността на „тип на раздела“ в „платен“ се визуализират бутони с наличните платени раздели. Не могат да се дублират еднакви платени раздели в един курс.

За момента има наличен само един платен раздел:



The form consists of three main components: a dropdown menu labeled 'Тип на раздела' (Section Type) with 'Платен раздел' (Paid section) selected; a text input field labeled 'Описание' (Description); and a button labeled 'Платени раздели:' (Paid sections:) with 'Истории' (Stories) selected. Below these is a green 'Добавяне' (Add) button.

Фиг. 59 Форма за добавяне на платен раздел

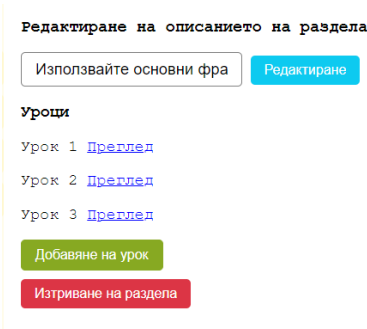
Ако добавянето на раздела е било успешно, се извежда съобщение известяващо за това, в противен случай се извежда съобщение за грешка.



The screenshot shows a green success message at the top: 'Успешно създаден раздел!' (Section created successfully!). Below it is the title 'Раздели' (Sections). On the left, there is a language selector 'Английски' (English) with a dropdown arrow. The main content is divided into two sections: 'Безплатни раздели' (Free sections) and 'Платени раздели' (Paid sections). Under 'Безплатни раздели', there is a list of seven sections: 'Раздел 1 - Използвайте основни фрази', 'Раздел 2 - Поздравявайте хората', 'Раздел 3 - Представете се', 'Раздел 4 - Използвайте сегашно време', 'Раздел 5 - Разходка из града', 'Раздел 6 - Членове на семейството', and 'Раздел 7 - Нов раздел'.

Фиг. 60 Успешно добавяне на раздел

Повече информация за разделите може да се разгледа при натискане на бутона за съответния раздел:



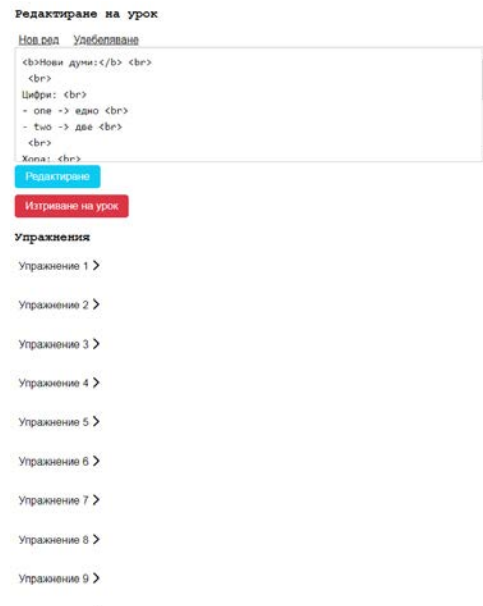
Фиг. 61 Преглед на раздел

Тъй като правилната последователност на разделите и уроците е от изключително важно значение за правилното проследяване на прогреса на потребителите, при изтриване на някои от тях е нужно да се промени и последователността на останалите и прогреса на потребителя да бъде актуализиран. Затова особено внимание трябва да се обърне на начина на запазване на данните за поредност на раздели и уроци още в проектирането на базата данни. За коригиране на тези данни се използват SQL транзакции, които са особено надежден метод, гарантиращ, че всяка колона в таблица, която трябва да се редактира в следствие на тази промяна, ще бъде успешно редактирана. Възникване на проблем при някои от тези звена ще предотврати коригирането на всяка свързана таблица към транзакцията, което ще гарантира запазване на коректността на данните.

Друго важно правило за бизнес логиката е да могат да се добавят уроци единствено към последния добавен раздел. По този начин ще се осигури правилна последователност на уроците и потребителите ще следват пътеката на обучение, без да се налага да се връщат назад към предишен раздел, за да отключат следващ урок, който в противен случай би могъл да се намира в съвсем случаен раздел. Това ще предотврати объркване от страна на потребителя и ще улесни работата на администраторите при поддържане на данните в системата.

В страницата за преглед на урок администраторите могат да преглеждат добавените упражнения към урока, както и да ги редактират или изтриват. Също така могат да добавят информация за урока, която да бъде достъпна през бутона за „Преглед“ от изскачащия прозорец в потребителската страница. За по-добро форматиране на текста за потребителската страница, е нужно да се добавят HTML тагове в колоните базата данни. За удобство на администраторите са добавени бутони за нов ред и удебеляване на текст над текстовото поле за попълване на информацията за урока, които да вмъкват нужните стилове в текста, там където се

намира курсора на мишката в текущия момент. Автоматизирайки този процес, администраторите не е нужно да пишат HTML тагове на ръка, както и да познават тези тагове.



Фиг. 62 Преглед на урок

Чрез бутона за добавяне на ново упражнение се отваря формата за добавяне, където да се въведе информацията за упражнението. Важно правилно при добавяне на упражненията е правилният отговор, записан в полета, да може да се получи от възможните варианти за отговор. Ако типът на отговора е „множествен отговор“, то тогава един от вариантите трябва да е изцяло еднакъв с правилния отговор. Ако е избран тип на отговора „съставяне на изречение“, тогава всяка дума от правилния отговор трябва да се съдържа в полетата за вариант за отговор. Това добавя допълнителна сигурност за коректността на въведените данни при добавяне на ново упражнение.

Фиг. 63 Добавяне на безплатно упражнение

Ако упражнението е добавено успешно, се извежда съобщения за успех, в противен случай администраторът вижда съобщение, че упражнението не е било добавено.

Упражненията могат да бъдат редактирани от прозореца за преглед на уроци, като формата за редактиране е идентична на тази за добавяне. При редактиране на упражнение отново се извършват същите проверки за коректност на въведените данни за „възможни отговори“ и „правилен отговор“.

Платените раздели и уроци следват подобна структура на организация на данните като безплатните раздели. Особеното при тях е, че между различните раздели няма връзка, тоест не следват една последователност. Единствено последователност се запазва за уроците към платените раздели, като отново при изтриване на урок се коригира последователността на останалите, които са добавени след него, както и прогреса на потребителите.

Добавянето на платени упражнения от тип „истории“ изисква работа с изображения. За да се гарантира достъпността на добавените снимки, те се запазват в директория, намираща се в проекта. За запазването на файловете в зададената директория се грижи мидълуеър, който се добавя като междинен слой към заявката за добавяне на упражнение. Генерира се случаен низ от символи, който се запазва като име на файла, за да се предотвратят конфликти при записването на изображението в директорията. Този низ от символи след това се изпраща към контролера и се запазва като име на файла в базата данни, който след това се селектира, за да бъде визуализиран в изгледите.

Към всяко изображение се запазва неговата последователност към дадения урок. Тази последователност се проверява при отговора на потребителя, когато преминава урока, за да се определи дали е подредил правилно снимките от историята. За да се гарантира, че тази последователност не е била подправена при изпращането на заявката, се извършват допълнителни проверки, гарантиращи, че изображенията ще бъдат добавени в правилната си последователност.

Задание

Добавяне на снимки

1 Choose File No file chosen

2 Choose File No file chosen

3 Choose File No file chosen

+

* До полето за добавяне на снимка се задава приоритетността на снимките

Добавяне

Фиг. 64 Добавяне на упражнение от раздел „истории“

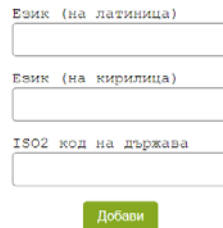
Редактирането на упражнение от този раздел се случва по два начина. Заданието се редактира по традиционния метод, по който се редактират останалите текстови полета за вход на данни. За редакция на добавените изображения се извършват операции изтриване и добавяне. Изтриването премахва перманентно снимката от базата данни, след което редактира последователността на останалите изображения. Добавянето може да се извърши само за една снимка наведнъж, за разлика от добавянето на ново упражнение, където могат да се добавят много на брой снимки. Новото изображение се добавя на последно място като последователност след останалите добавени изображения.

Добавянето на администраторски профили може да се изпълнява единствено от главните администратори. Съответно бутонът за добавяне в навигационната секция е видима единствено от профили с тези привилегии. За допълнителна защита се прави и проверка в backend частта, за да се гарантира, че няма да могат да се изпращат заявки към endpoint-а чрез други методи.

Добавянето на нов език се извършва през формата за добавяне, достъпна чрез бутона в навигационната лента. Администраторите е нужно да въведат информация за езика и ISO2 кода на държавата, където се говори, за да се генерира флаг, видим в

потребителските страници. Ако се въведе грешна стойност за ISO2 код, се извежда грешка и добавянето не е успешно.

Добавяне на език



Език (на латиница)

Език (на кирилица)

ISO2 код на държава

Добави

Фиг. 65 Добавяне на нов език



Грешка в ISO2 код!

Избор на език

Език (на латиница)

Език (на кирилица)

ISO2 код на държава

Добави

Фиг. 66 Грешни данни за ISO2 код

Всички операции с данните за магазина могат да се извършват през страницата за преглед на предмети в магазина. Могат да се редактират или изтриват съществуващи предмети, както и да се добавят нови.

Абонаменти

Количество

7

дни

Цена

1000

лв.

Монети

Редактиране

Изтриване

Количество

30

дни

Цена

7.99

лв.

Пари

Редактиране

Изтриване

Монети

Количество

500

монети

Цена

4.99

лв.

Редактиране

Изтриване

Добавяне на предмет в магазина

Добавяне

Количество

Цена

Тип на продукта

Монети

Тип на плащане

Монети

Добавяне

Фиг. 67 Форма за добавяне на предмет в магазина

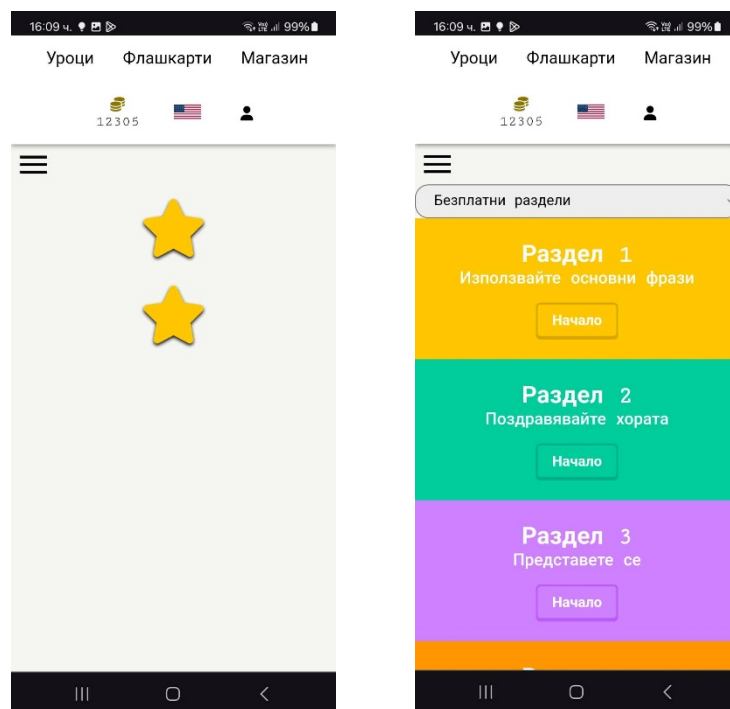
Фиг. 68 Преглед на магазин

3.3. Responsive дизайн

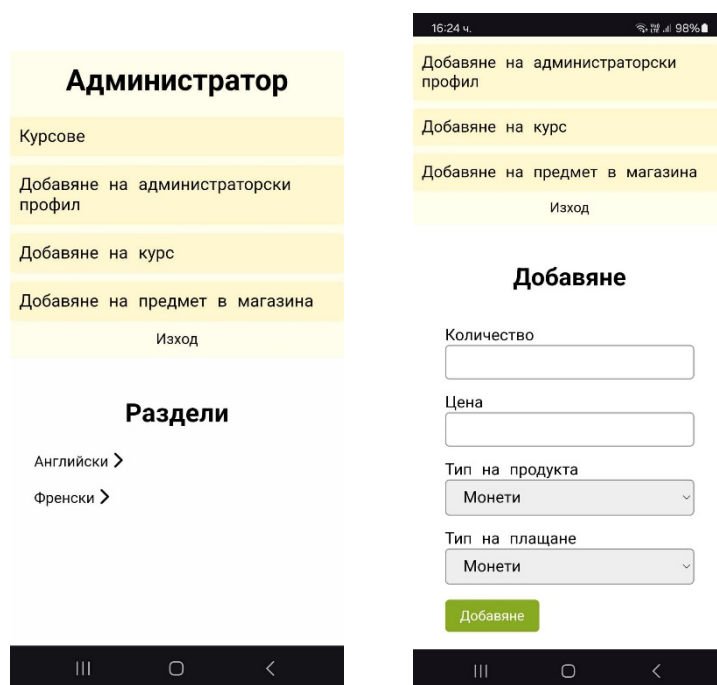
Уеб приложението поддържа responsive дизайн, както за потребителската страница, така и за администраторския панел.

Responsive дизайнът е много важна част от уеб разработката, който гарантира, че уеб приложението се адаптира и функционира безпроблемно на различни устройства и размери на екрана. По същество той позволява на един уебсайт да осигури оптимално изживяване при гледане и взаимодействие, независимо дали е достъпен от настолен компютър, лаптоп, таблет или смартфон.

Понеже целта на разработеното приложение е то да бъде достъпно за голям брой хора, имплементирането на дизайн, функциониращ за всички видове устройства, е задължително при разработката на приложението.



Фиг. 69 Дизайн на потребителска страница



Фиг. 70 Дизайн на администраторски панел

Глава V – Тестове и резултати

При реализацията на дипломната работа се използва автоматично тестване с цел проверка на правилната функционалност основни компоненти на системата.

Unit тестването е критична практика в разработката на софтуер, която има за цел да провери коректността на отделни единици или компоненти на програма. Тези единици могат да бъдат функции, методи или класове, тествани изолирано, за да се гарантира, че се държат според очакванията. Този тип тестване предоставя няколко предимства, включително подобро качество на кода, по-бързо отстраняване на грешки и повишена увереност при промени в кода.

В текущата реализация са използвани следните Node.js модули, служещи за извършване на Unit тестване – `mocha`, `chai` и `sinon`.

Mocha е универсална и популярна JavaScript тестова рамка, използвана за писане и изпълнение на тестове. Библиотеката осигурява стабилна и гъвкава среда за организиране и провеждане на тестове, като поддържа както синхронно, така и асинхронно тестване. Структурата на Mocha позволява на разработчиците да дефинират тестови пакети и индивидуални тестови случаи, като използват описателни функции като `describe()` и `it()`, подобрявайки четливостта и поддръжката на тестовия код. Освен това Mocha се интегрира добре с различни библиотеки като Chai.

Chai е библиотека, правеща извършените тестове четливи и изразителни. Библиотеката предлага няколко стила на утвърждаване, включващи `expect`, `should` и `assert`, което позволява на разработчиците да изберат стил, който отговаря на техните предпочитания и стил на кодиране.

Sinon допълва Mocha и Chai, като предоставя мощни инструменти за `mocking`, `stubbing` и `spying` на функции и обекти. Инструментът `mocking` позволява на разработчиците да симулират сложно поведение или зависимости, изолирайки тествания модул от външни влияния. Инструментът `stubbing` заменя функциите с предвидимо поведение, улеснявайки контролирано тестване на взаимодействията. Инструментът `spying` наблюдава извикванията на функции, предоставяйки представа за това как функциите се използват по време на изпълнение на теста. Възможностите на Sinon са от съществено значение за тестване на сценарии, включващи асинхронни операции, мрежови заявки или интеграции на трети страни.

Изпълнени са следните тестови сценарии:

1. Създаване на профил

Създаването на профил изисква няколко важни проверки за осигуряване на правилното функциониране на приложението. Преминават се през следните стъпки, които проверяват коректността на данните:

- Проверка за съществуващ потребител със същото потребителско име или имейл;
- Проверка за съвпадение на полетата за „парола“ и „повтаряне на парола“;
- Проверка за коректността на въведените данни в полетата за потребителско име и парола. Тестват се Regex функции за определяне на тяхната коректност. След успешно преминаване на този тест се извършва хеширане на паролата;
- Проверка за възникване на обща грешка при изпълнение на функциите за добавяне на потребител.

На база тези проверки се създаде тест, който изведе следните резултати:

```
Account Controller - createUser
  ✓ should redirect to lessons if user registration is successful
  ✓ should redirect to register with error flash if username or email is taken
  ✓ should redirect to register with error flash if username or password validation fails
  ✓ should redirect to register with error flash if passwords do not match
Error: Something went wrong
    at Context.<anonymous> (D:\DiplomnaRabota\src\tests\registrationTests.js:75:60)
    at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:366:21)
    at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
    at Runner.runTest (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:677:10)
    at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:800:12
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:592:14)
    at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:602:7
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:485:14)
    at cbHookRun (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:550:7)
    at done (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:310:5)
    at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:389:7)
    at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:509:10)
    at Immediate.<anonymous> (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:570:5)
    at process.processImmediate (node:internal/timers:478:21)
  ✓ should redirect to register with error flash if there is an exception

5 passing (12ms)
```

Фиг. 71 Резултат от тест за добавяне на потребител

2. Вход в профил

За правилното вписване на потребител в системата се преминава през следните проверки:

- Проверка за съществуване на потребител по въведено потребителско име;
- Проверка за съвпадение на въведената парола с тази, записана в базата данни. Преди да се извърши проверката, хешираната парола, получена от базата, се декриптира, след което се сравнява с въведената във формата за вход;
- Проверка за възникване на обща грешка при изпълнение на функциите за вход на потребител.

На база тези проверки се създаде тест, който изведе следните резултати:

```
Account Controller - loginUser
  ✓ should redirect to lessons if user login is successful and user type is 2
Error: User login fails
  at Context.<anonymous> (D:\DiplomnaRabota\src\tests\loginTests.js:47:59)
  at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:366:21)
  at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
  at Runner.runTest (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:677:10)
  at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:800:12
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:592:14)
  at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:602:7
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:485:14)
  at cbHookRun (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:550:7)
  at done (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:310:5)
  at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:389:7)
  at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:509:10)
  at Immediate.<anonymous> (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:570:5)
  at process.processImmediate (node:internal/timers:478:21)
  ✓ should redirect to login with error flash if user login fails
Error: Something went wrong
  at Context.<anonymous> (D:\DiplomnaRabota\src\tests\loginTests.js:56:59)
  at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:366:21)
  at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
  at Runner.runTest (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:677:10)
  at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:800:12
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:592:14)
  at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:602:7
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:485:14)
  at cbHookRun (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:550:7)
  at done (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:310:5)
  at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:389:7)
  at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:509:10)
  at Immediate.<anonymous> (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:570:5)
  at process.processImmediate (node:internal/timers:478:21)
  ✓ should redirect to login with error flash if there is an exception

3 passing (12ms)
```

Фиг. 72 Резултат от тест за вход на потребител

3. Добавяне на курс

За правилното добавяне на нов език е нужно да се извършат следните проверки:

- Проверка за вече съществуващ език;
- Проверка за коректно въведен ISO2 код на държавата, в която се говори даденият език;
- Проверка за възникване на обща грешка при изпълнение на функциите за вход на потребител.

На база тези проверки се създаде тест, който изведе следните резултати:

```
Courses Controller - addCourse
  ✓ should successfully add a course and redirect with success message
TypeError: Cannot read properties of undefined (reading 'toUpperCase')
    at e (D:\DiplomnaRabota\node_modules\country-flags-svg\dist\index.cjs:3:25608)
    at t (D:\DiplomnaRabota\node_modules\country-flags-svg\dist\index.cjs:3:25844)
    at Array.find (<anonymous>)
    at m (D:\DiplomnaRabota\node_modules\country-flags-svg\dist\index.cjs:3:25646)
    at t (D:\DiplomnaRabota\node_modules\country-flags-svg\dist\index.cjs:3:25844)
    at Object.addCourse (D:\DiplomnaRabota\src\controllers\coursesController.js:43:26)
    at Context.<anonymous> (D:\DiplomnaRabota\src\tests\addCourseTests.js:42:33)
    at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:366:21)
    at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
    at Runner.runTest (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:677:10)
    at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:800:12
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:592:14)
    at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:602:7
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:485:14)
    at cbHookRun (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:550:7)
    at done (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:310:5)
    at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:389:7)
    at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:509:10)
    at Immediate._onImmediate (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:570:5)
    at process.processImmediate (node:internal/timers:478:21)
  ✓ should be a none existing course
  ✓ should handle error when ISO2 code is invalid
Error: Something went wrong
    at Context.<anonymous> (D:\DiplomnaRabota\src\tests\addCourseTests.js:59:58)
    at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:366:21)
    at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
    at Runner.runTest (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:677:10)
    at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:800:12
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:592:14)
    at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:602:7
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:485:14)
    at cbHookRun (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:550:7)
    at done (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:310:5)
    at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:389:7)
    at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
    at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:509:10)
    at Immediate._onImmediate (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:570:5)
    at process.processImmediate (node:internal/timers:478:21)
  ✓ should handle general error during course addition

4 passing (17ms)
```

Фиг. 73 Резултат от тест за добавяне на нов език

4. Добавяне на урок

За добавяне на ново упражнение е нужно извършването на проверка за въведените варианти за отговор и въведеният правилен отговор. Ако чрез вариантите за отговор не може да се достигне до правилният отговор, то тогава упражнението няма да бъде добавено успешно.

Резултатите от този тест демонстрираха различните сценарии, които ще се изпълнят от полученият резултат от заявката:

```
Exercises Controller - addExercise
  ✓ should add an exercise and redirect to lesson page if successful
  ✓ should redirect with error if exercise data is invalid
Error: Something went wrong
  at Context.<anonymous> (D:\DiplomnaRabota\src\tests\addExerciseTests.js:60:23)
  at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:366:21)
  at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
  at Runner.runTest (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:677:10)
  at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:800:12
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:592:14)
  at D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:602:7
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:485:14)
  at cbHookRun (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:550:7)
  at done (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:310:5)
  at callFn (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:389:7)
  at Runnable.run (D:\DiplomnaRabota\node_modules\mocha\lib\runnable.js:354:5)
  at next (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:509:10)
  at Immediate.<anonymous> (D:\DiplomnaRabota\node_modules\mocha\lib\runner.js:570:5)
  at process.processImmediate (node:internal/timers:478:21)
  ✓ should handle exceptions and redirect with error

3 passing (13ms)
```

Фиг. 74 Резултат от тест за добавяне на урок

Глава VI – Заключение

За създаването на интерактивно и правилно функциониращо приложение за изучаване на чужди езици е необходимо разработването както на динамичен и приятен за използване интерфейс, така и на бизнес логика, гарантираща правилното взаимодействие между отделните компоненти в системата.

За тази цел в представената дипломна работа беше наблегнато еднакво на клиентската и на сървърната част. А чрез внимателно проектиране на системата за съхранение на данни се осигури плавната и безпроблемна работа на приложението, гарантиращо гъвкавост при извършване на всякакви видове корекции на съществуващите данни.

Достъп до уеб приложението имат широк кръг от потребители, благодарение на съвместимостта си към всички типове устройства. По този начин всеки ще има възможността да обогатява своите знания навсякъде и по всяко време.

Разработеното приложение може да се развие в няколко направления:

- Създаване на мобилно приложение;
- Създаване на повече разнообразие за видове упражнения и уроци;
- Добавяне на изкуствен интелект за подобряване на обучението.

Списък с фигури

Фиг. 1 Лого на Node.js.....	11
Фиг. 2 Лого на HTML и CSS.....	12
Фиг. 3 Лого на EJS.....	13
Фиг. 4 Лого на JavaScript	14
Фиг. 5 Лого на MySQL.....	14
Фиг. 6 Лого на Stripe	15
Фиг. 7 Архитектура на системата	18
Фиг. 8 Use Case диаграма за потребителска страница	21
Фиг. 9 Use Case диаграма за администраторска страница.....	22
Фиг. 10 Flowchart диаграма за потребителска страница	23
Фиг. 11 Flowchart диаграма за вход на администратор	24
Фиг. 12 Flowchart диаграма за преглед на курсове, раздели, уроци и упражнения	24
Фиг. 13 Flowchart диаграма за функционалности на администратор	25
Фиг. 14 Концептуален модел на БД.....	29
Фиг. 15 Протопит на страница за вход	31
Фиг. 16 Прототип на страница за визуализиране на уроци	31
Фиг. 17 Прототип на страница за визуализиране на упражнения	32
Фиг. 18 Прототип на страница за флашкарти	32
Фиг. 19 Прототип на администраторски панел.....	33
Фиг. 20 Релационна база от данни	35
Фиг. 21 Табл. user_types.....	36
Фиг. 22 Табл exercise_task_types	39
Фиг. 23 Табл. exercise_option_types	39
Фиг. 24 Табл. shop_item_types.....	43
Фиг. 25 Табл payment_types.....	44
Фиг. 26 Структура на проекта	44
Фиг. 27 Начален екран на приложението	50
Фиг. 28 Избор на курс при регистрация.....	50
Фиг. 29 Въвеждане на данни за регистрация.....	50
Фиг. 30 Форма за вход.....	52
Фиг. 31 Съобщение за грешка при вход	52
Фиг. 32 Екран за преглед на безплатни раздели и уроци	53
Фиг. 33 Изскачащ прозорец за начало на урок	53
Фиг. 34 Преглед на урок	54
Фиг. 35 Изскачащ прозорец за начало на урок	54
Фиг. 36 Урок от тип четене и съставяне на изречение	55
Фиг. 37 Урок от тип слушане и множествен отговор	55
Фиг. 38 Грешен отговор	56
Фиг. 39 Правилен отговор	56
Фиг. 40 Преглед на платени раздели.....	57
Фиг. 41 Упражнение на платен раздел „Истории“	57
Фиг. 42 Правилен отговор	58
Фиг. 43 Преглед на падащо меню за курсове.....	59
Фиг. 44 Страница за стартиране на нов курс	59
Фиг. 45 Падащо меню за започнати курсове.....	60
Фиг. 46 Магазин	60
Фиг. 47 Форма за плащане с банкова карта.....	61
Фиг. 48 Преглед на извършени покупки.....	62
Фиг. 49 Страница за преглед на флашкарти.....	62
Фиг. 50 Добавяне на категория	63
Фиг. 51 Редактиране на категория.....	63
Фиг. 52 Изтриване на категория.....	63
Фиг. 53 Добавяне на флашкарта.....	64
Фиг. 54 Препоръчани флашкарти	64
Фиг. 55 Преглед на флашкарта.....	65

Фиг. 56 Преглед на курсове	66
Фиг. 57 Преглед на раздели	66
Фиг. 58 Форма за добавяне на безплатен раздел	66
Фиг. 59 Форма за добавяне на платен раздел.....	67
Фиг. 60 Успешно добавяне на раздел.....	67
Фиг. 61 Преглед на раздел	68
Фиг. 62 Преглед на урок	69
Фиг. 63 Добавяне на безплатно упражнение.....	70
Фиг. 64 Добавяне на упражнение от раздел „истории“	71
Фиг. 65 Добавяне на нов език.....	72
Фиг. 66 Грешни данни за ISO2 код	72
Фиг. 68 Форма за добавяне на предмет в магазина	73
Фиг. 67 Преглед на магазин.....	73
Фиг. 69 Дизайн на потребителска страница.....	74
Фиг. 70 Дизайн на администраторски панел.....	74
Фиг. 71 Резултат от тест за добавяне на потребител	76
Фиг. 72 Резултат от тест за вход на потребител.....	77
Фиг. 73 Резултат от тест за добавяне на нов език	78
Фиг. 74 Резултат от тест за добавяне на урок	79

Използвани източници

[1] Node.js v22.3.0 documentation - <https://nodejs.org/docs/latest/api/>

[2] REST API Design Best Practices Handbook – How to Build a REST API with JavaScript, Node.js, and Express.js - <https://www.freecodecamp.org/news/rest-api-design-best-practices-build-a-rest-api/>

Jean-Marc Möckel – 4.05.2022

[3] Express.js API documentation - <https://expressjs.com/en/5x/api.html>

[4] Stripe documentation - <https://docs.stripe.com/api>

[5] NodeJS Unit Testing: Getting Started Tutorial - <https://www.lambdatest.com/learning-hub/nodejs-unit-testing>

Solomon Eseme – 13.03.2023

[6] Bcrypt API - <https://www.npmjs.com/package/bcrypt>

[7] EJS documentation - <https://ejs.co/#docs>

[8] JavaScript documentation - <https://developer.mozilla.org/en-US/docs/Web/JavaScript>

[9] Storyteller – снимки и текст за платен раздел в приложението

[10] Font Awesome икони - <https://fontawesome.com/start>

Приложение А: Исходен код

➤ index.js

```
const path = require("path");
const express = require("express");
const methodOverride = require('method-override');
const session = require('express-session');
const flash = require('connect-flash');
require("dotenv").config();
const app = express();
app.use(methodOverride('_method'));
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
app.use(express.static(__dirname + '/public'));
app.use(session({
  name: process.env.SESSION_NAME,
  resave: false,
  saveUninitialized: false, // Will not save empty sessions
  secret: process.env.SESSION_SECRET,
  cookie: {
    maxAge: 7 * 24 * 60 * 60 * 1000, // One week
    httpOnly: true, // Prevents client side scripts from accessing data
    sameSite: true
  }
}));
app.use(flash());
app.set('views', path.join(__dirname, 'views'));
app.set('view engine', 'ejs');
const coursesRouter = require('./routes/coursesRoutes');
const sectionsRouter = require('./routes/sectionsRoutes');
const adminRouter = require('./routes/adminRoutes');
```